

# 10-423/623/723: Generative AI

## Lecture 17 – Distributed Training

Matt Gormley & Aran Nayebi

(Slide credits: Henry Chai & Pat Virtue)

## Llama-1

When training a 65B-parameter model, our code processes around 380 tokens/sec/GPU on 2048 A100 GPU with 80GB of RAM. This means that training over our dataset containing 1.4T tokens takes approximately 21 days.

## Llama-2

|         |     | Time<br>(GPU hours) | Power<br>Consumption (W) | Carbon Emitted<br>(tCO <sub>2</sub> eq) |
|---------|-----|---------------------|--------------------------|-----------------------------------------|
| LLAMA 2 | 7B  | 184320              | 400                      | 31.22                                   |
|         | 13B | 368640              | 400                      | 62.44                                   |
|         | 34B | 1038336             | 350                      | 153.90                                  |
|         | 70B | 1720320             | 400                      | 291.42                                  |
| Total   |     | 3311616             |                          | 539.00                                  |

## Llama-3

**Compute.** Llama 3 405B is trained on up to 16K H100 GPUs, each running at 700W TDP with 80GB HBM3, using Meta’s Grand Teton AI server platform (Matt Bowman, 2022). Each server is equipped with eight GPUs and two CPUs. Within a server, the eight GPUs are connected via NVLink. Training jobs are scheduled

Recall: How much did it cost to train LLaMa?



- “the newly announced clusters both contain 24,576 Nvidia Tensor Core H100 GPUs. This is a significant increase over the original clusters, which contained 16,000 Nvidia A100 GPUs.”

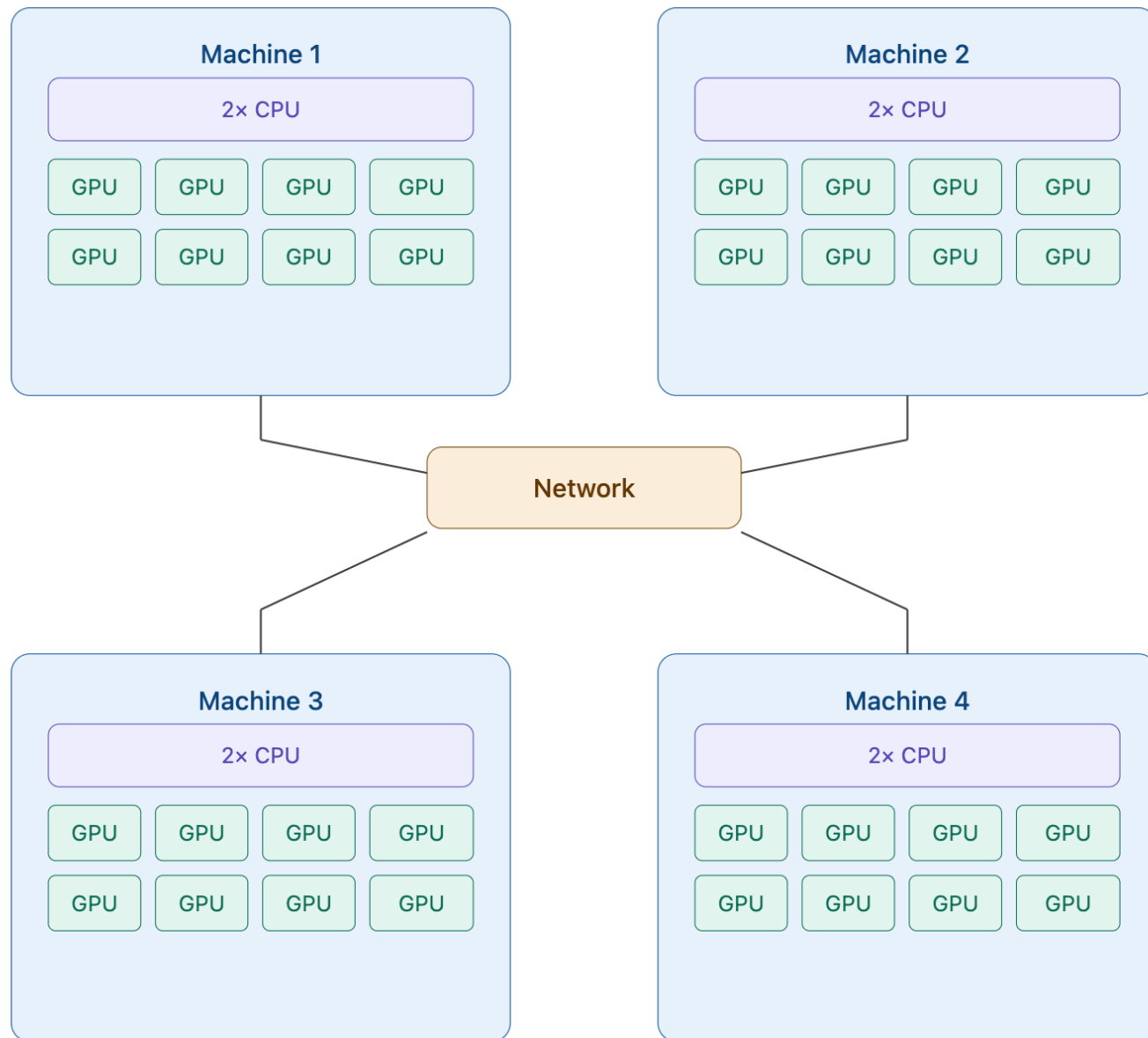
Recall: How much did it cost to train LLaMa?

# GPU Comparison

|                       | NVIDIA A100       | NVIDIA H100       | NVIDIA B200       |
|-----------------------|-------------------|-------------------|-------------------|
| Memory size           | 80 GB             | 80 GB             | 192 GB            |
| BF16 Performance      | 0.6 PFLOPS        | 1.9 PFLOPS        | 4.5 PFLOPS        |
| Peak memory bandwidth | 1.6 TB / second   | 1.6 TB / second   | 8 TB / second     |
| Inter-GPU bandwidth   | ~ 0.6 TB / second | ~ 0.9 TB / second | ~ 1.8 TB / second |
| Price                 | ~ \$10K           | ~ \$25K           | ~\$50k            |

- Key takeaway: inter-GPU communication is the primary bottleneck in distributed systems (and speeding it up is worth a lot of money!)

# A (Tiny) High Performance Computing Cluster



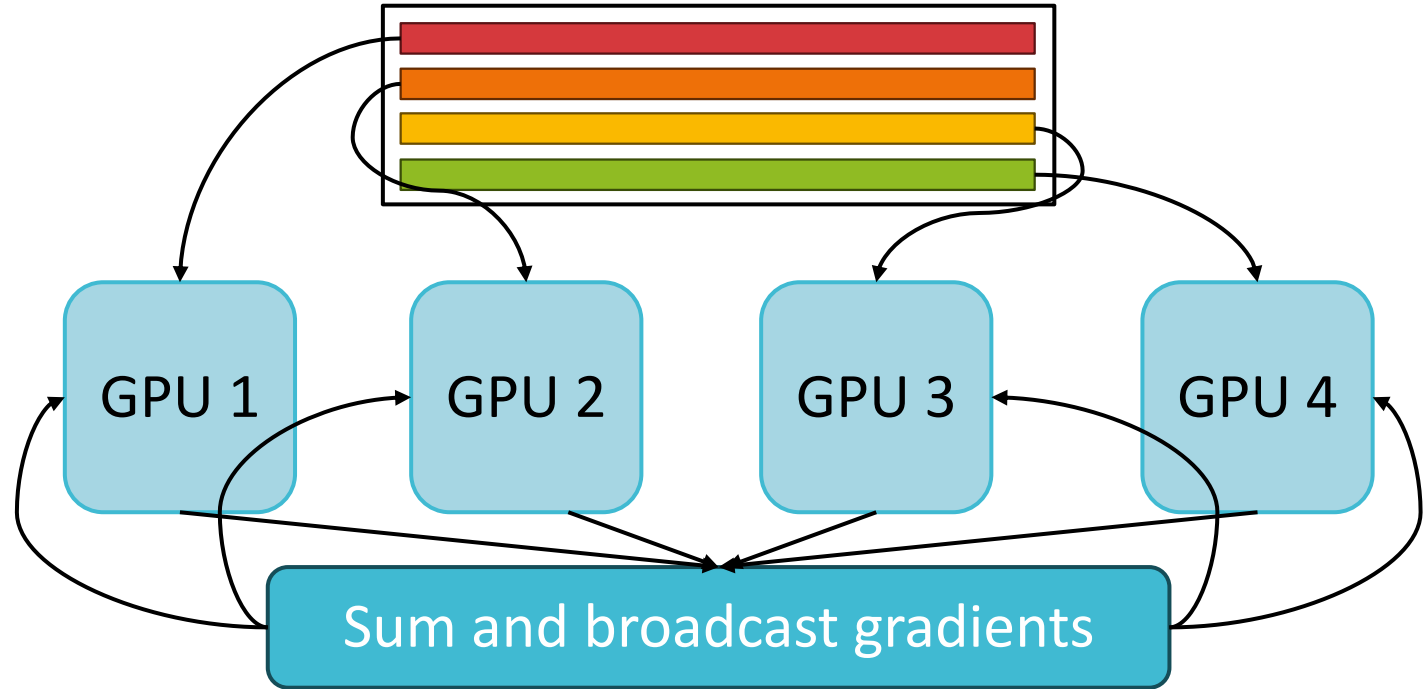
# Parallelism in LLM Training

- Goal: divide the work of training an LLM across multiple GPUs such that inter-GPU communication is minimized
- Good news: Transformer-based LLM architectures are highly parallelizable!
- Easily exploitable parallelism in LLM training includes:
  - Data parallelism
  - Model or tensor parallelism
  - Pipeline parallelism
  - Optimizer-based parallelism
  - Token parallelism
  - Expert parallelism



## Data Parallelism

- Approach: during training, split each minibatch of data points evenly across multiple GPUs and have each GPU compute the forward and backward pass for its data points.



- The simplest, most effective form of parallelism *if attainable*
  - Only one inter-GPU communication per iteration

**Question:** What is the fundamental problem here?

## Llama-4

### Llama 4 Behemoth

**288B** active parameter, **16** experts  
**2T** total parameters

The most intelligent teacher model for distillation

### Llama 4 Maverick

**17B** active parameters, **128** experts  
**400B** total parameters

Native multimodal with **1M** context length

### Llama 4 Scout

**17B** active parameters, **16** experts  
**109B** total parameters

Industry leading **10M** context length  
Optimized inference

# How big is LLaMa-4?



## Maverick:

- If parameters are stored in BF16, each parameter takes 16 bits = 2 bytes ...
- ... which means 400 billion parameters requires 800 billion bytes = ~800 GB!
- But our GPUs only have 192 GB of RAM
- Idea: split the model up across multiple GPUs!

## Llama-4

### Llama 4 Behemoth

288B active parameter, 16 experts  
2T total parameters

The most intelligent teacher model for distillation

### Llama 4 Maverick

17B active parameters, 128 experts  
400B total parameters

Native multimodal with 1M context length

### Llama 4 Scout

17B active parameters, 16 experts  
109B total parameters

Industry leading 10M context length  
Optimized inference

# How big is LLaMa-4?

## *Behemoth:*

- If parameters are stored in BF16, each parameter takes 16 bits = 2 bytes ...
- ... which means 2 trillion parameters requires 4 trillion bytes = ~4,000 GB!
- But our GPUs only have 192 GB of RAM
- Idea: split the model up across multiple GPUs!

## **Llama-4**

### **Llama 4 Behemoth**

**288B** active parameter, **16** experts  
**2T** total parameters

The most intelligent teacher model for distillation

### **Llama 4 Maverick**

**17B** active parameters, **128** experts  
**400B** total parameters

Native multimodal with **1M** context length

### **Llama 4 Scout**

**17B** active parameters, **16** experts  
**109B** total parameters

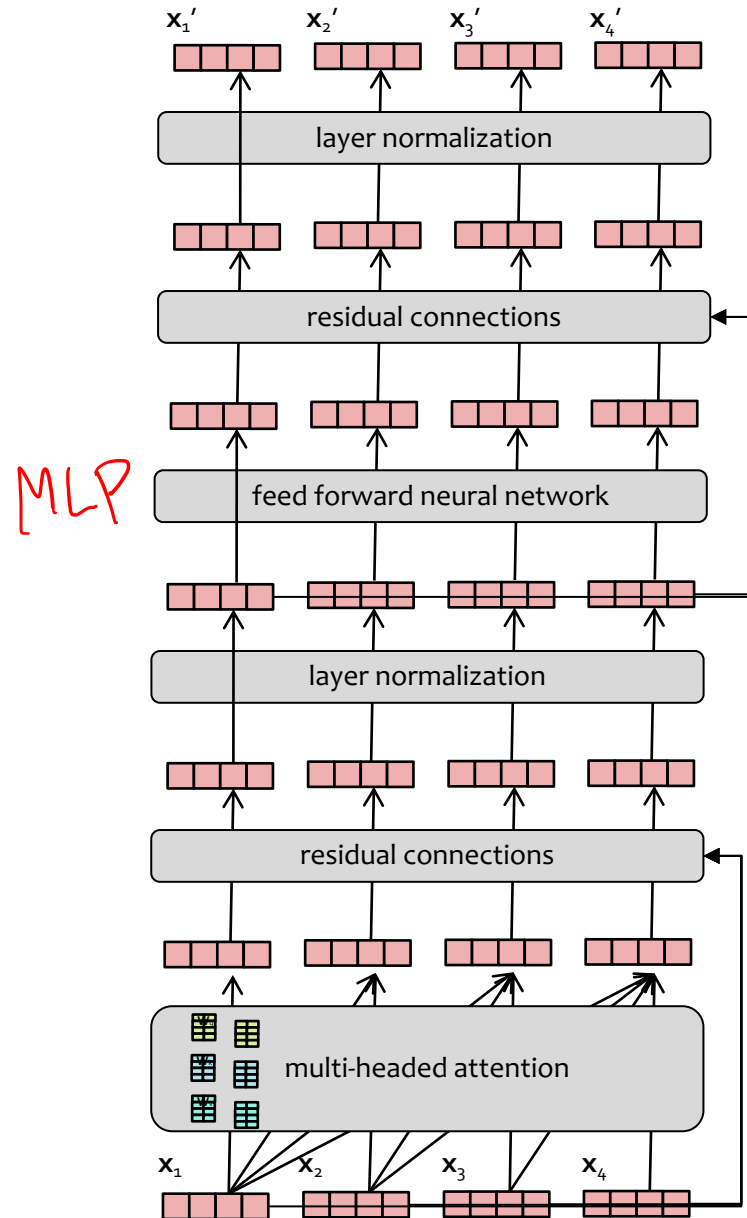
Industry leading **10M** context length  
Optimized inference

# How big is LLaMa-4?

# Model Parallelism

- Approach: for a batch of data points, partition the forward and backward computations *within a layer* across multiple GPUs
  - Also called tensor parallelism
- Transformer based architectures have two primary modules that can be parallelized
  1. MLP blocks
  2. Attention blocks

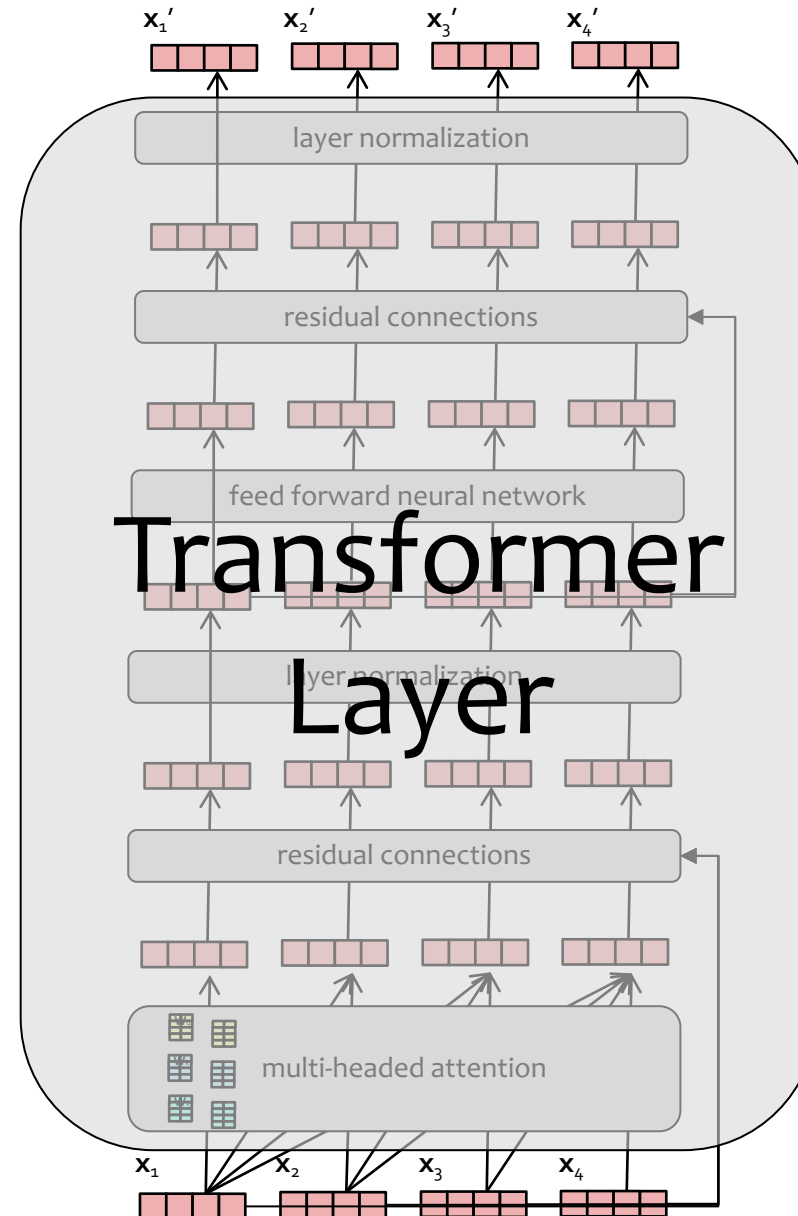
# Transformer Layer



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

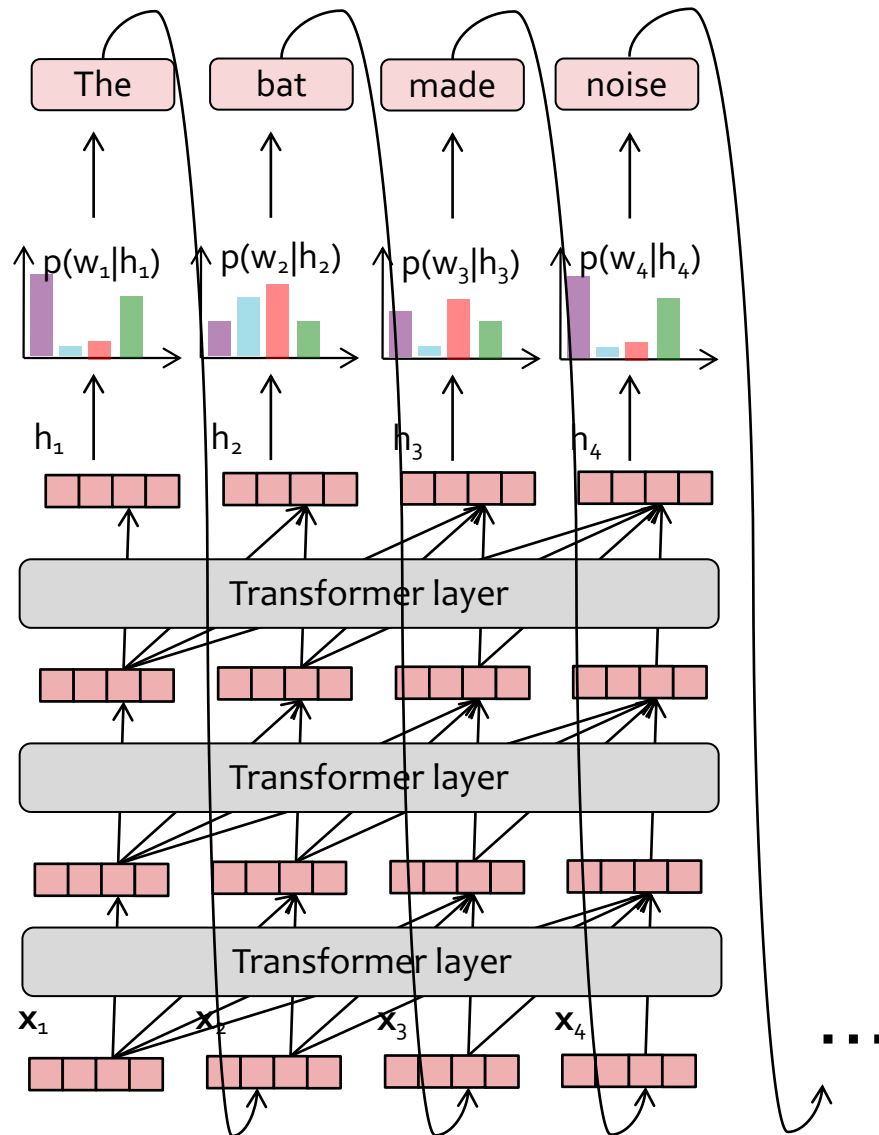
# Transformer Layer



**Each layer** of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

# Transformer Language Model



**Each layer** of a Transformer LM consists of several **sublayers**:

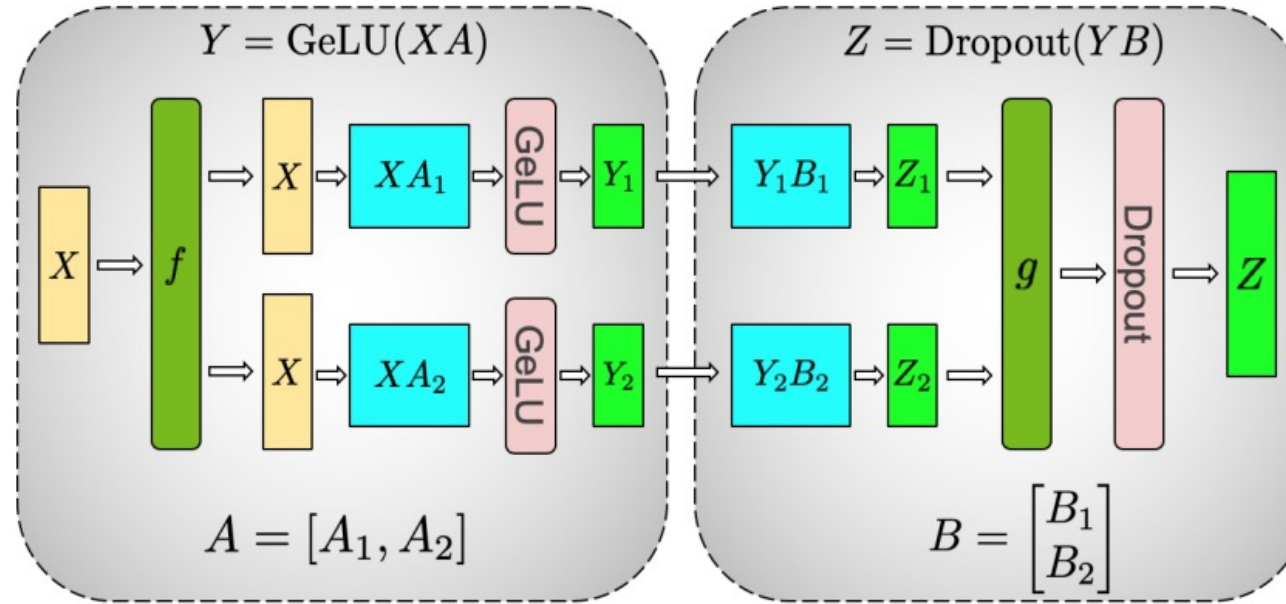
1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections





# Model Parallelism

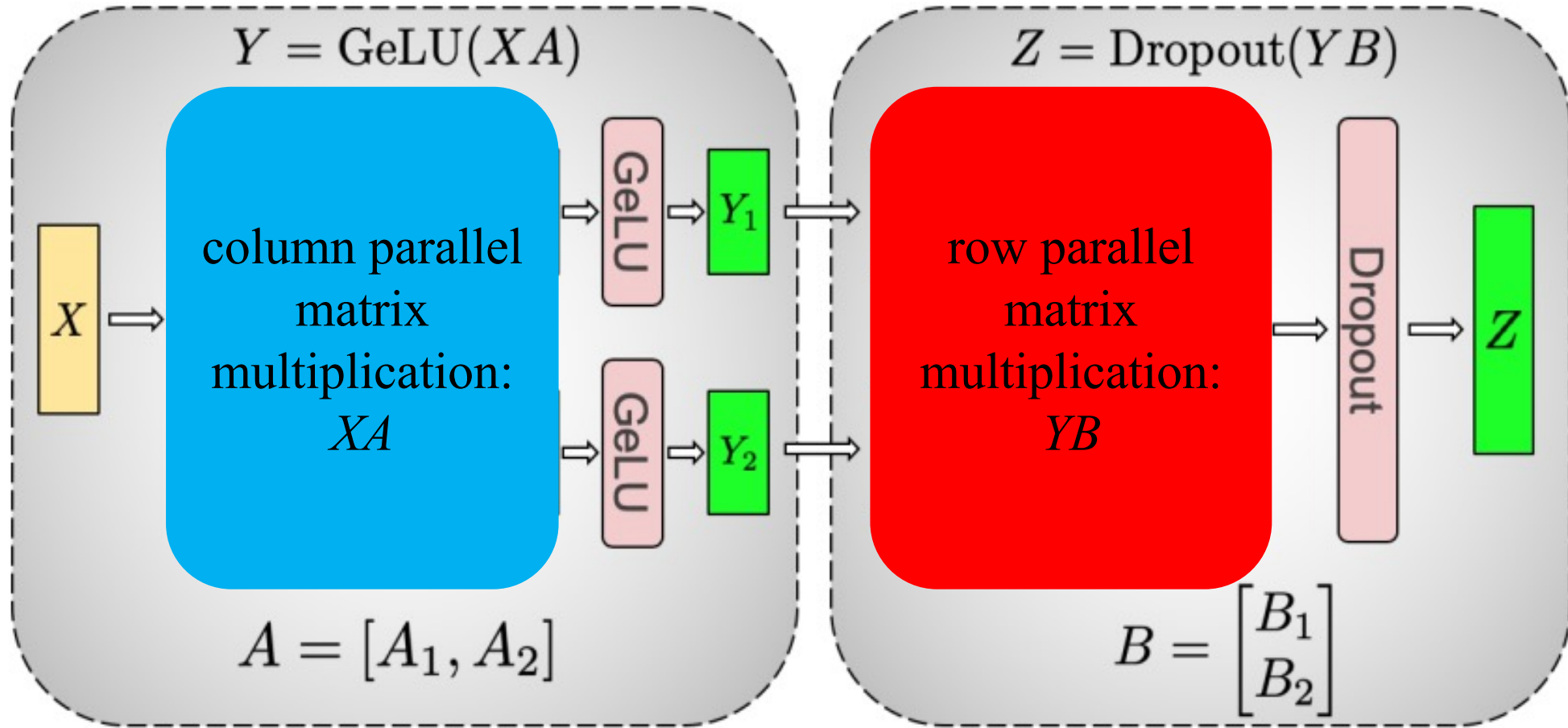
# Model Parallelism: MLP Blocks



(a) MLP

*Figure 3. Blocks of Transformer with Model Parallelism.  $f$  and  $g$  are conjugate.  $f$  is an identity operator in the forward pass and all reduce in the backward pass while  $g$  is an all reduce in the forward pass and identity in the backward pass.*

# Model Parallelism: MLP Blocks



(a) MLP

# Model Parallelism: MLP Blocks

Given a batch of training data points  $X \in \mathbb{R}^{N \times D}$ , and weight  $W \in \mathbb{R}^{D \times M}$  the operations to parallelize a Linear layer the MLP blocks

- $f(X) = XW$  in the forward pass and
- $\partial \ell / \partial X = GW^T$  in the backward pass,  $G = \partial \ell / \partial f$

Two ways of partitioning  $W$  across multiple GPUs

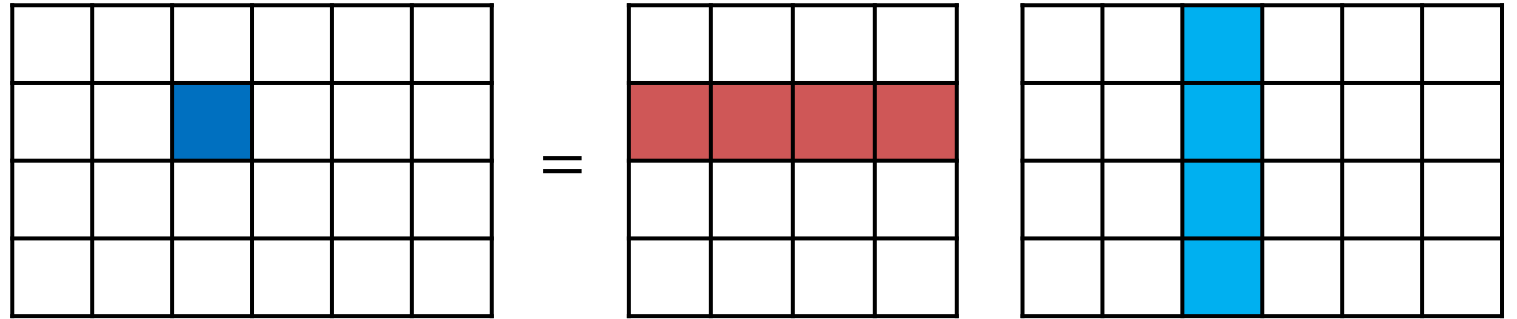
- Column-parallel:  $W = [c_1, c_2, \dots, c_M]$
- Row-parallel:  $W = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_D \end{bmatrix}$

# Aside: Parallel Matrix Multiplication

- Standard:

$$Z = XW$$

$$Z_{i,j} = \sum_k X_{i,k} W_{k,j}$$

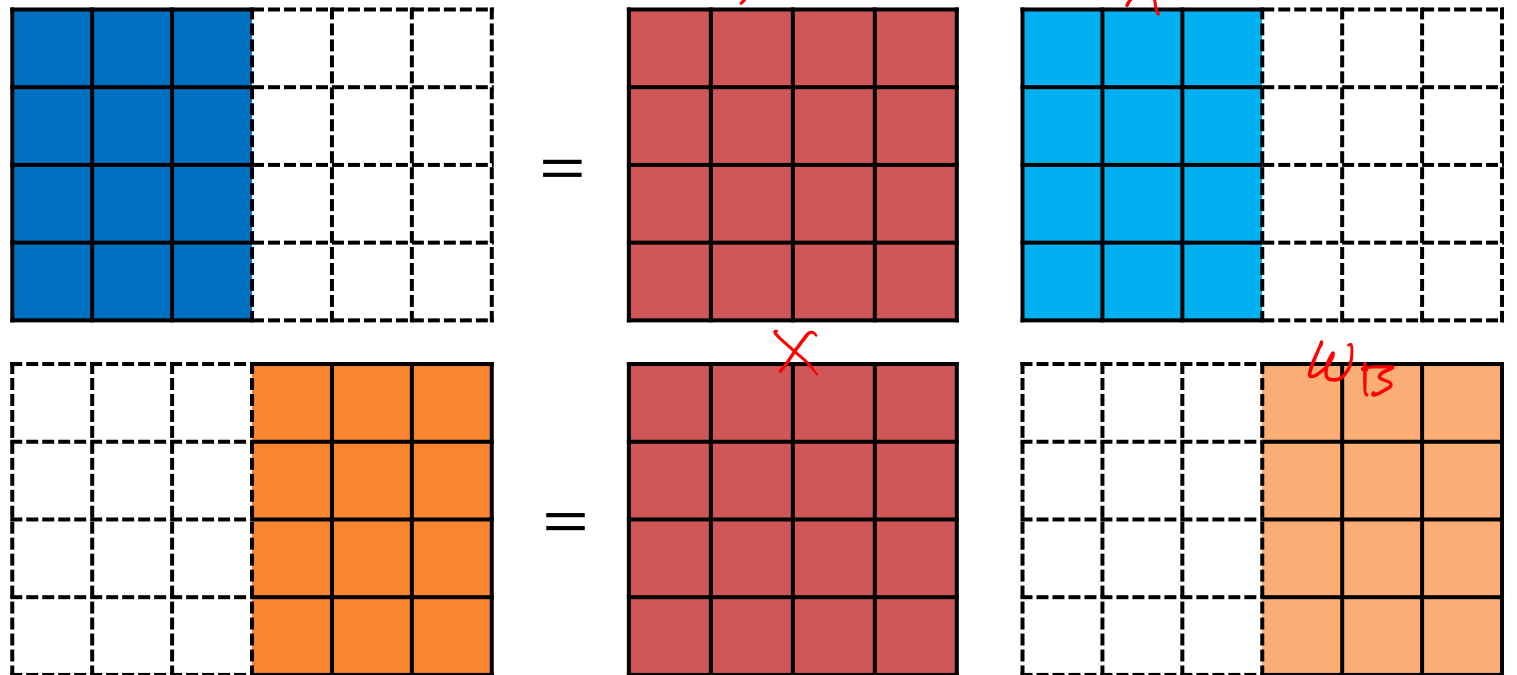
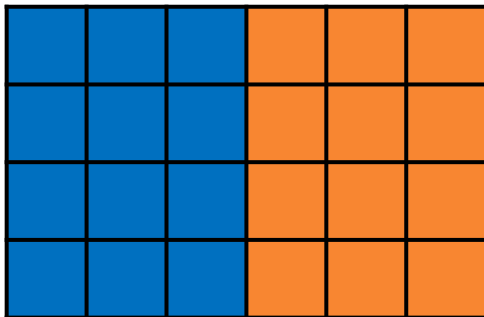


- Split  $W$  by column:

$$Z_A = XW_A$$

$$Z_B = XW_B$$

$$Z = [Z_A, Z_B]$$

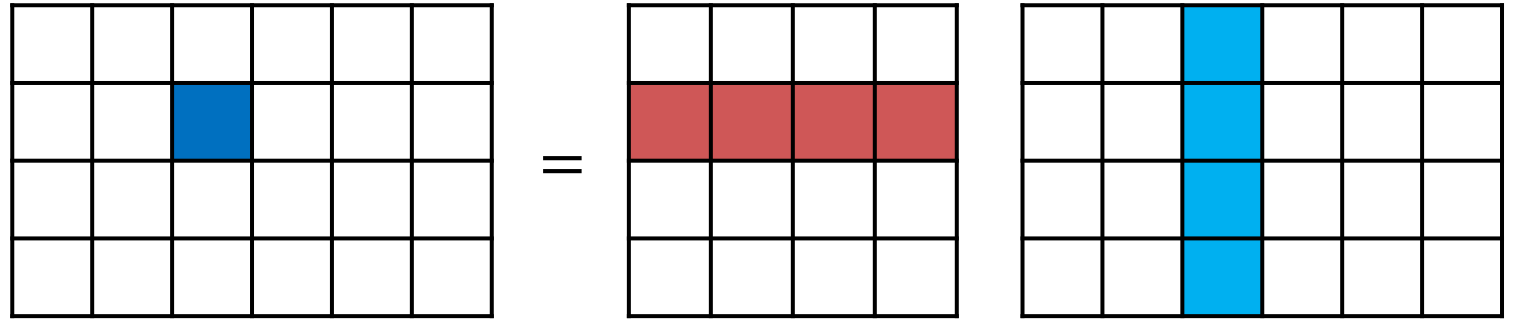


# Aside: Parallel Matrix Multiplication

- Standard:

$$Z = XW$$

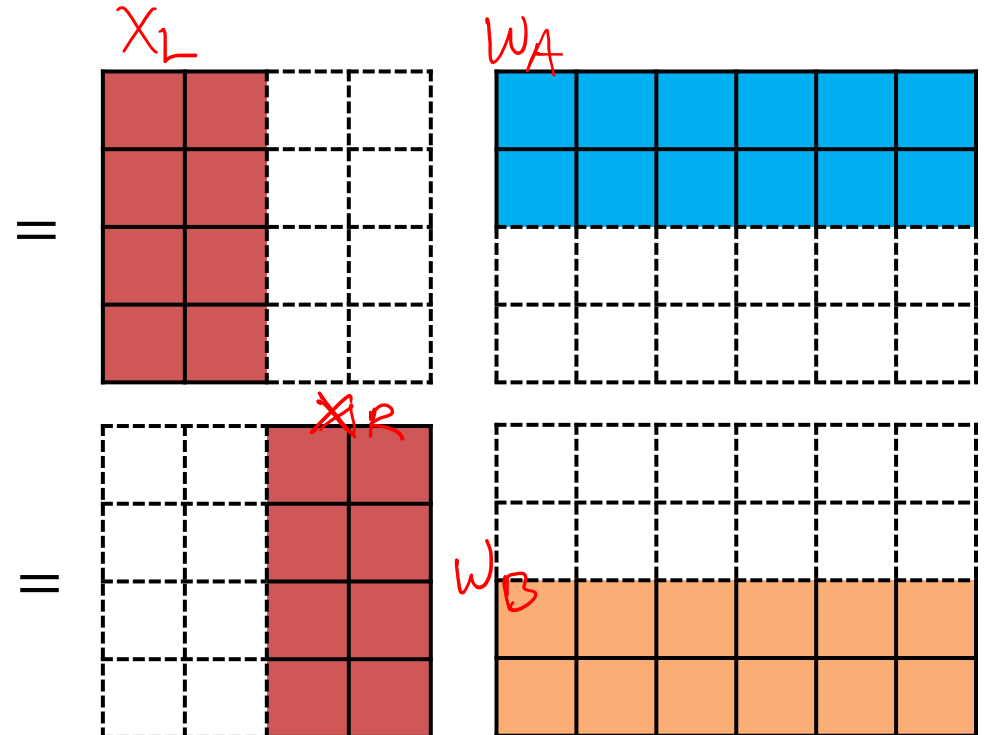
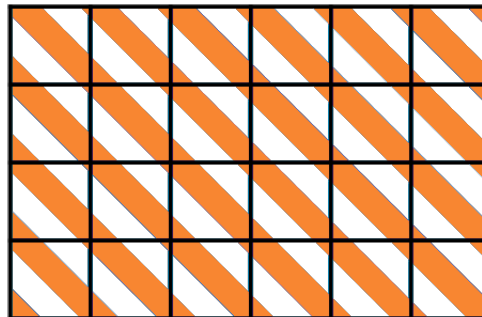
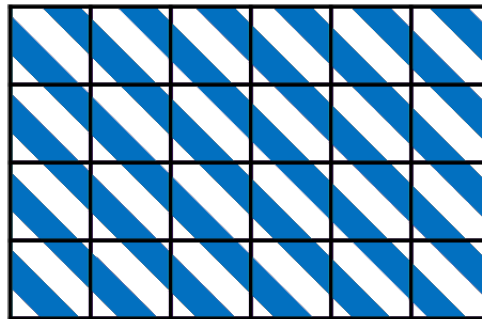
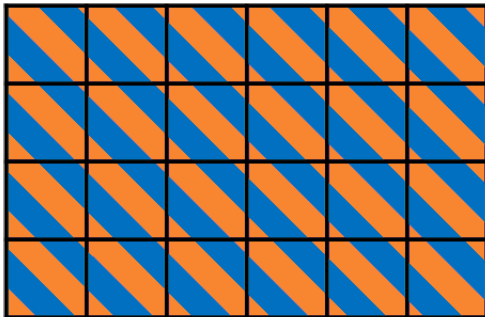
$$Z_{i,j} = \sum_k X_{i,k} W_{k,j}$$



- Split W by row:

$$Z_A = X_L W_A \quad Z = Z_A + Z_B$$

$$Z_B = X_R W_B$$



## Model Parallelism: MLP Blocks

Column-parallel:  $W = [c_1, c_2, \dots, c_M]$

- Forward pass:  $XW = [Xc_1, \dots, Xc_M]$  ← col par.

- $Y = XW$  in the forward pass
- $\partial \ell / \partial X = GW^T$  in the backward pass  
where  $G = \partial \ell / \partial Y$

- Backward pass:  $GW^T = [g_1, g_2, \dots, g_M] \begin{bmatrix} c_1^T \\ c_2^T \\ \vdots \\ c_M^T \end{bmatrix} = \sum_{m=1}^M g_m c_m^T$  ← row par.

Row-parallel:  $W = \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_D^T \end{bmatrix}$

- Forward pass:  $\underline{XW} = [x_1, x_2, \dots, x_D] \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_D^T \end{bmatrix} = \sum_{d=1}^D x_d r_d^T$  ← row par.

- Backward pass:  $GW^T = [Gr_1, \dots, Gr_D]$  ← col par.

# Model Parallelism: MLP Blocks

Column-parallel:  $W = [c_1, c_2, \dots, c_M]$

- Forward pass:  $XW = [Xc_1, \dots, Xc_M]$

- Backward pass:  $GW^T = [g_1, g_2, \dots, g_M] \begin{bmatrix} c_1^T \\ c_2^T \\ \vdots \\ c_M^T \end{bmatrix} = \sum_{m=1}^M g_m c_m^T$

Row-parallel:  $W = \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_D^T \end{bmatrix}$

- Forward pass:  $XW = [x_1, x_2, \dots, x_D] \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_D^T \end{bmatrix} = \sum_{d=1}^D x_d r_d^T$

- Backward pass:  $GW^T = [Gr_1, \dots, Gr_D]$

- $Y = XW$  in the forward pass
- $\partial \ell / \partial X = GW^T$  in the backward pass where  $G = \partial \ell / \partial Y$

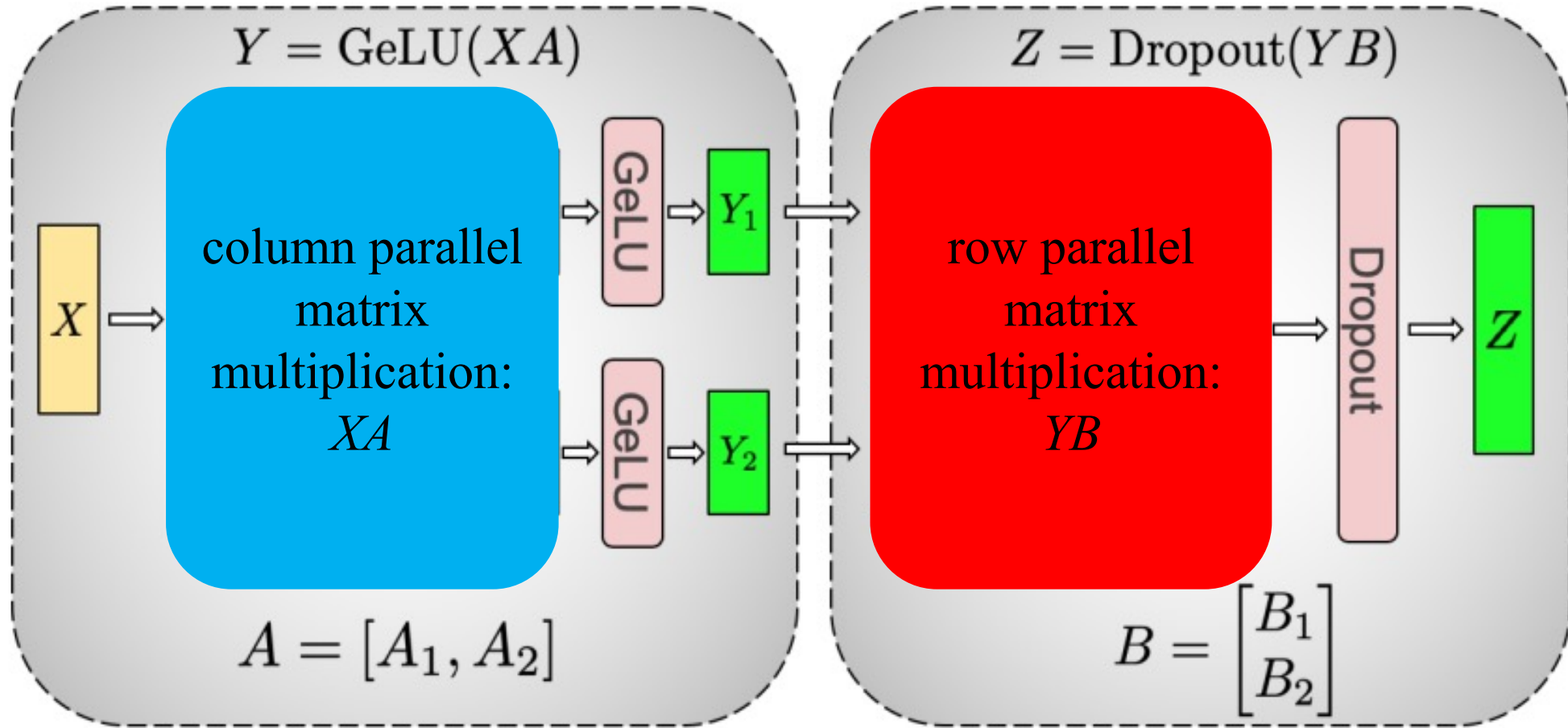
## Two key observations:

1) The column-parallel and row-parallel forward and backward computations are identical, just reversed

2) Consecutive MLPs can alternate between column-parallel and row-parallel implementations to minimize the sums across GPUs!



# Model Parallelism: MLP Blocks

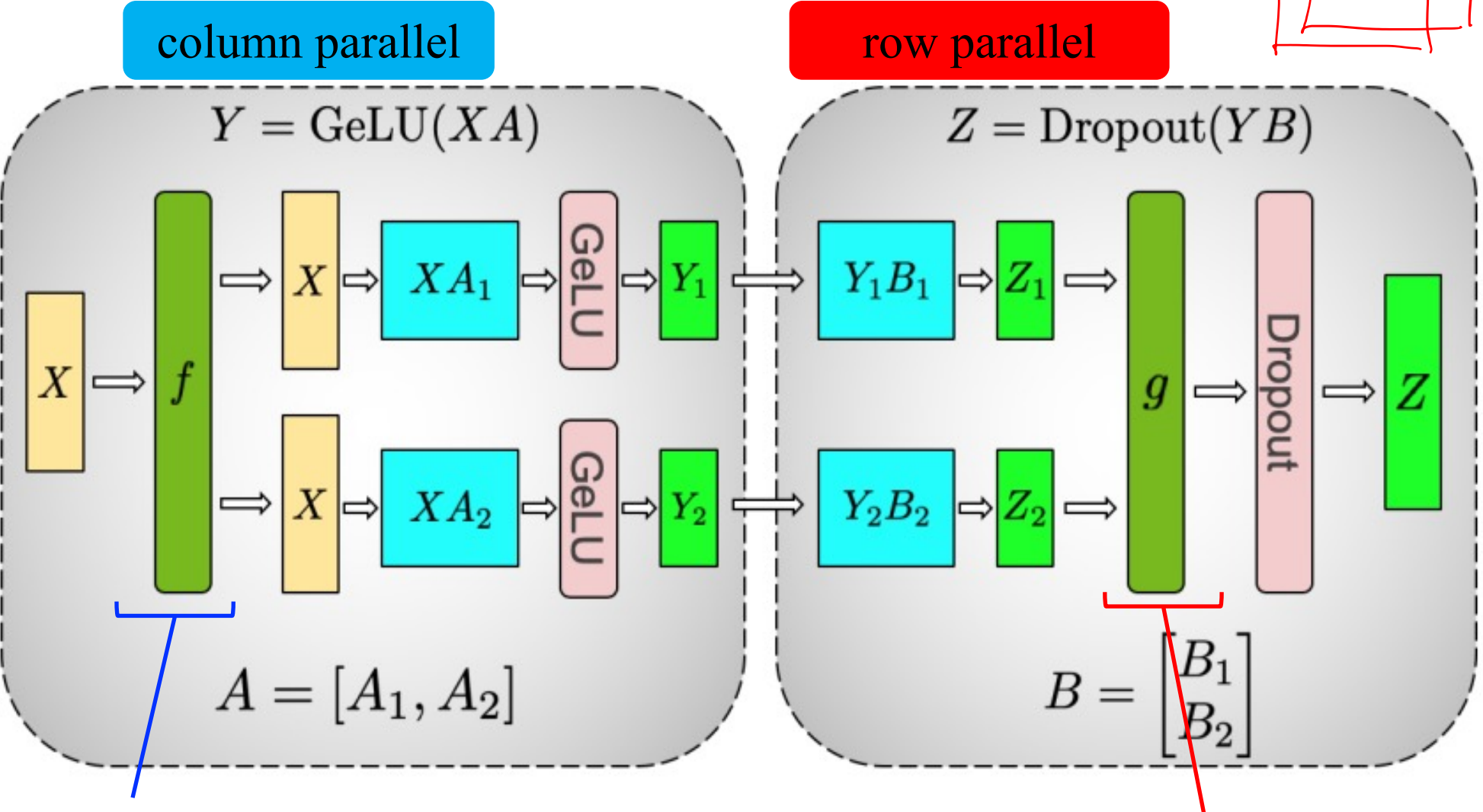


(a) MLP

Y  
 $\begin{bmatrix} Y_1 & Y_2 \end{bmatrix}$

# Model Parallelism: MLP Blocks

$Z_1^{Z_2}$   
 $Z$

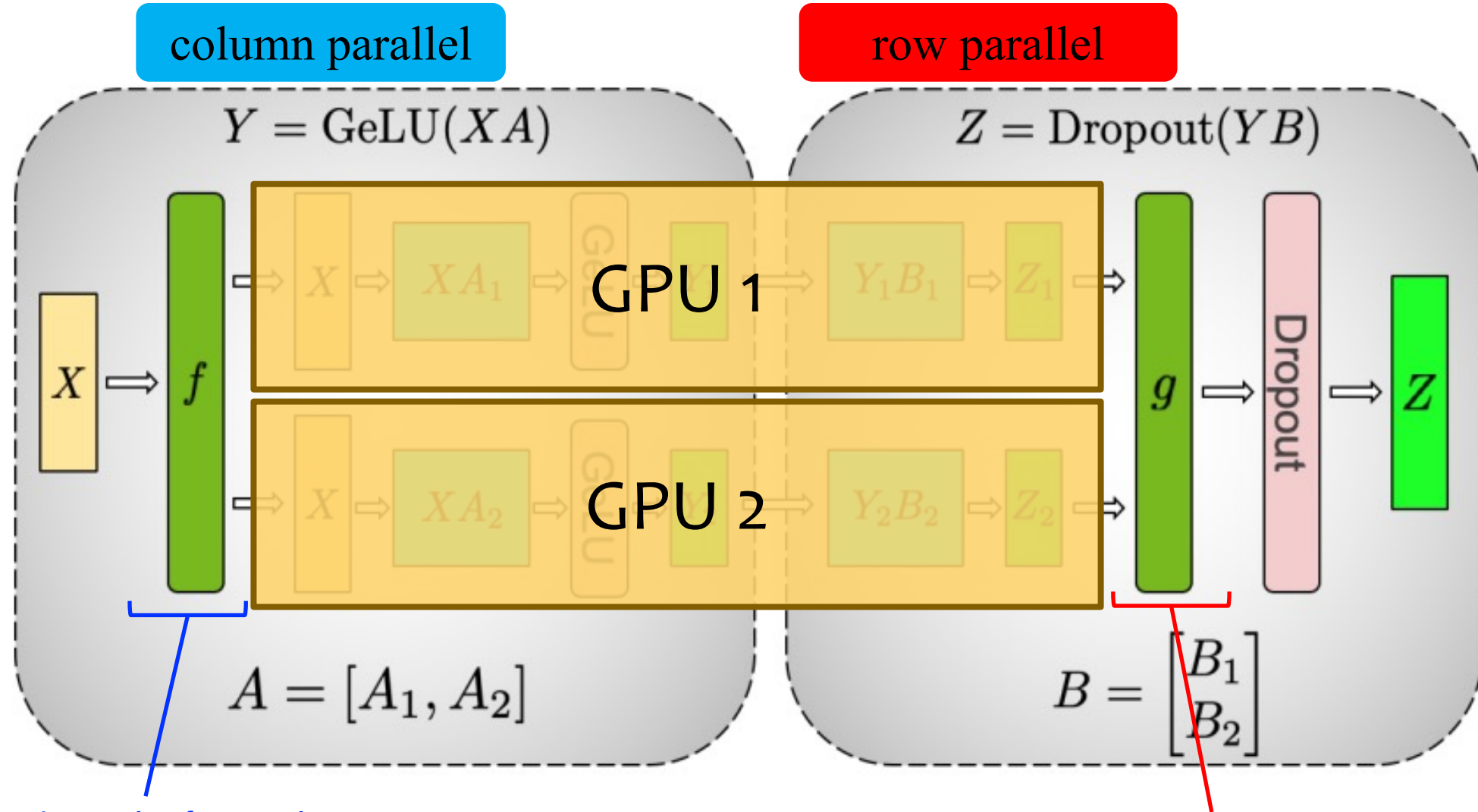


$f$  is **identity** in the forward pass  
 and **summation** in the backward pass

(a) MLP

$g$  is **summation** in the forward pass  
 and **identity** in the backward pass

# Model Parallelism: MLP Blocks



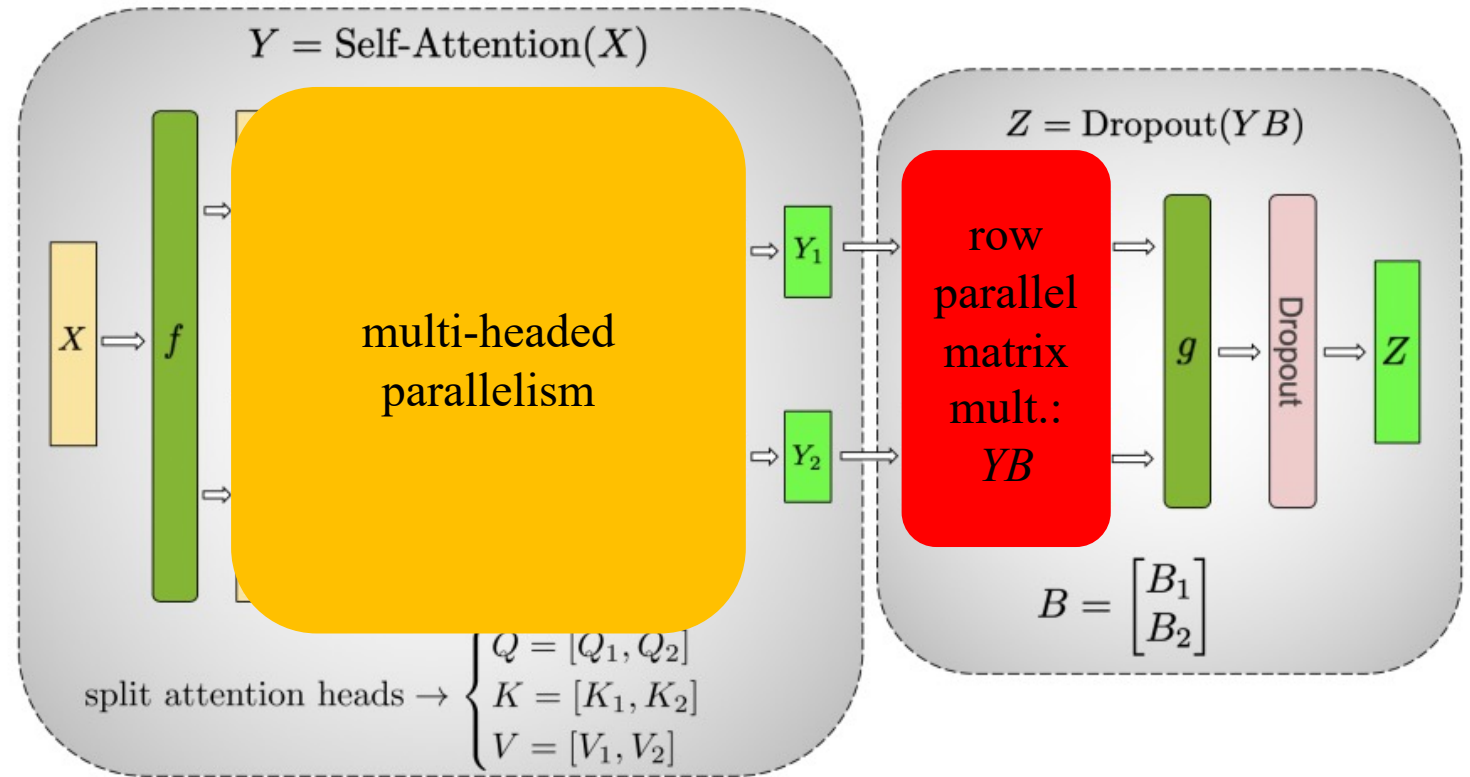
$f$  is **identity** in the forward pass  
and **summation** in the backward pass

(a) MLP

$g$  is **summation** in the forward pass  
and **identity** in the backward pass

# Model Parallelism: Attention Blocks

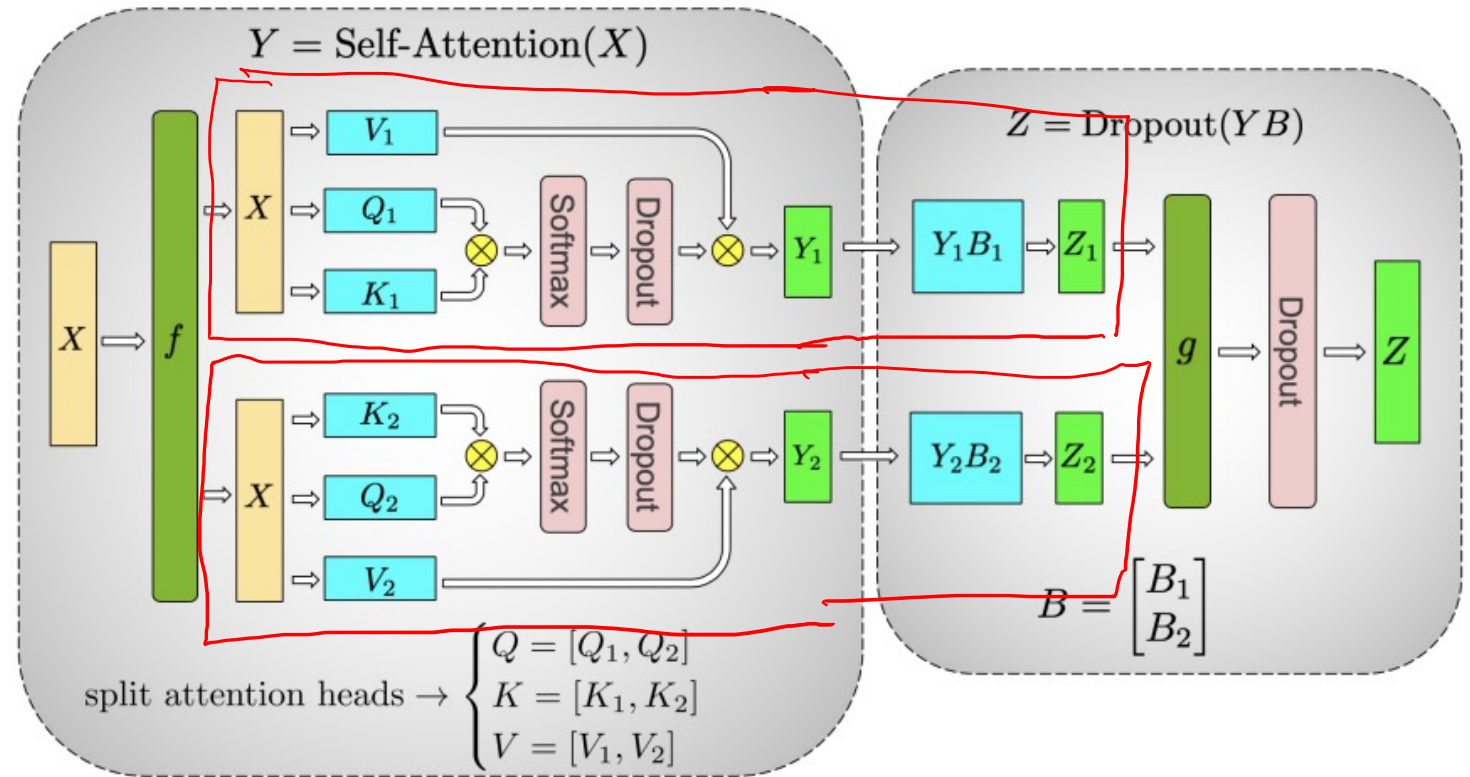
- Multi-headed attention blocks trivially parallelize across attention heads
- Assuming attention heads are concatenated horizontally, the output can be passed to a row-parallel MLP directly



**(b) Self-Attention**

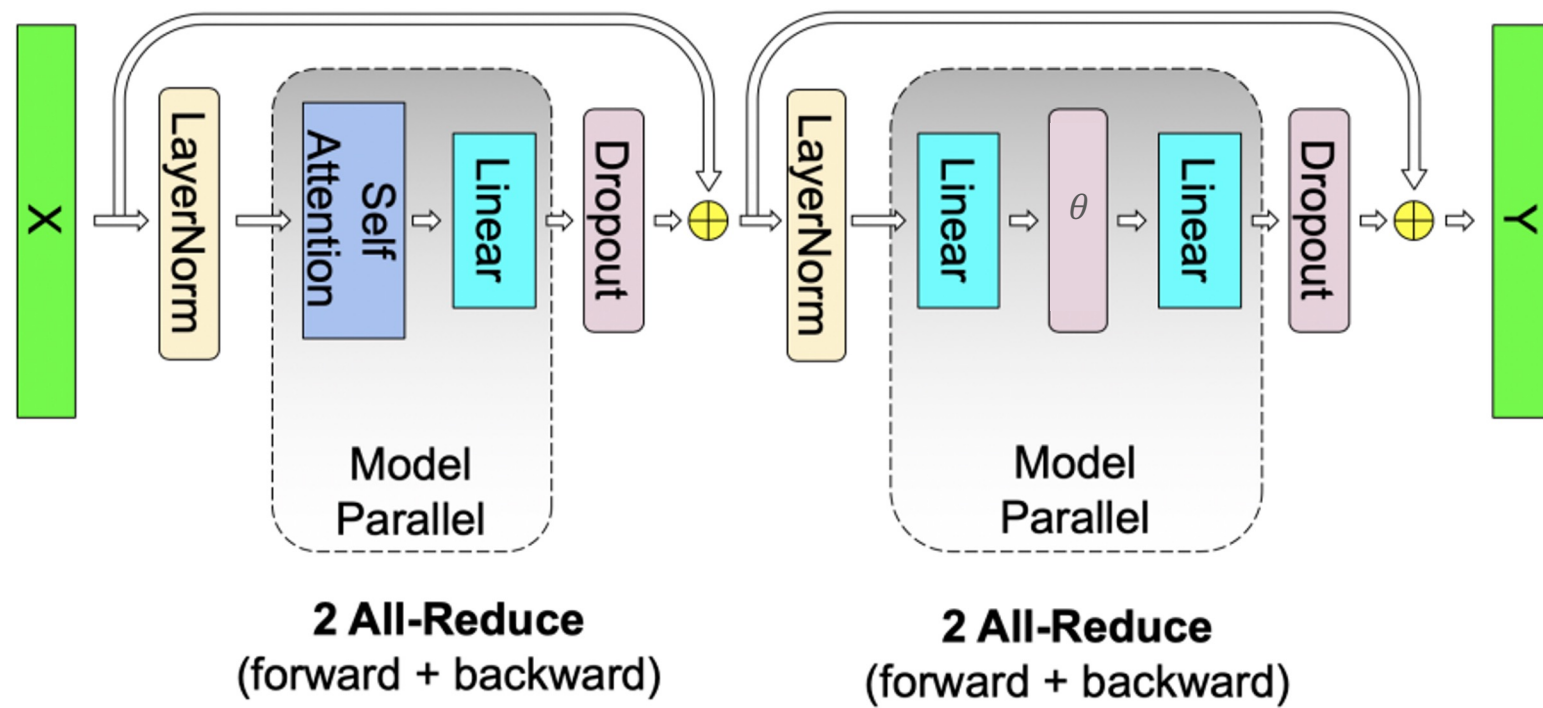
# Model Parallelism: Attention Blocks

- Multi-headed attention blocks trivially parallelize across attention heads
- Assuming attention heads are concatenated horizontally, the output can be passed to a row-parallel MLP directly



(b) Self-Attention

# Model Parallelism: Attention Blocks



*Figure 4.* Communication operations in a transformer layer. There are 4 total **communication** operations in the forward and backward pass of a single model parallel transformer layer.

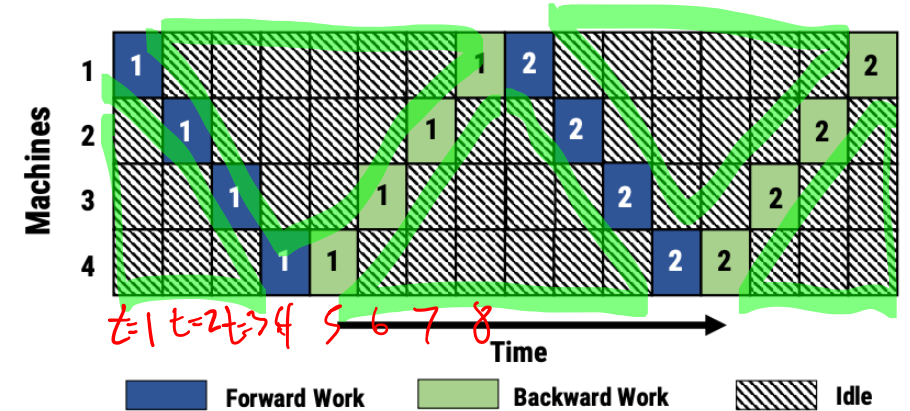
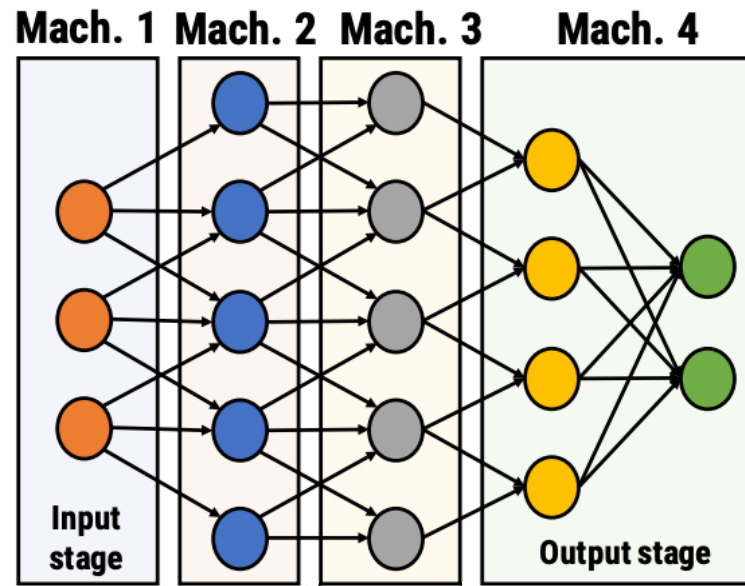


# Pipeline Parallelism



# Pipeline Parallelism

- The previous examples of tensor parallelism also implicitly leverage *pipeline parallelism*, where different layers or modules are put on different GPUs
- Issue: the computation across layers is inherently **sequential**, not parallel!

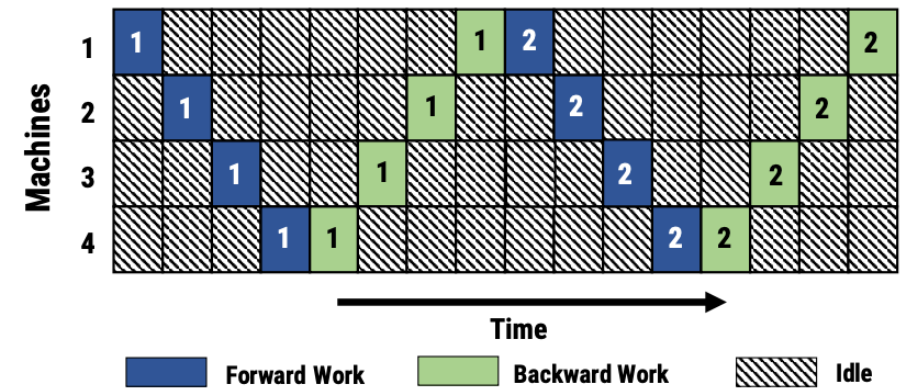
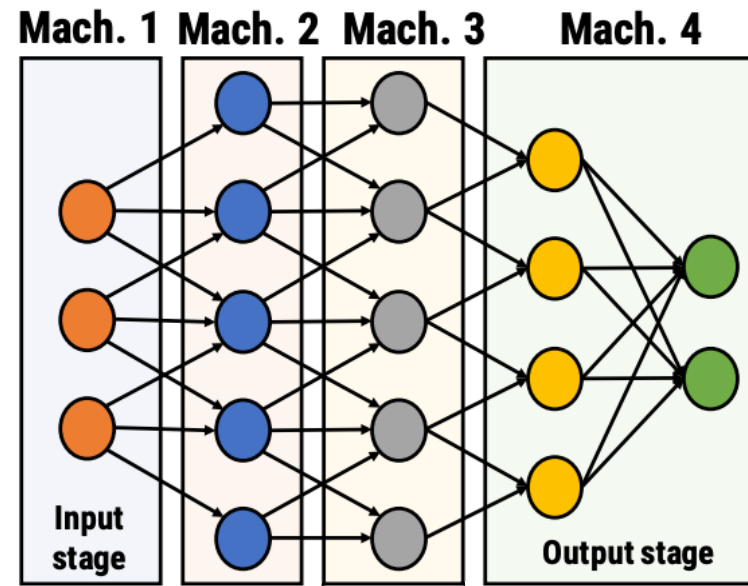


- Naïve implementation has a ton of idle GPU time!



# Pipeline Parallelism

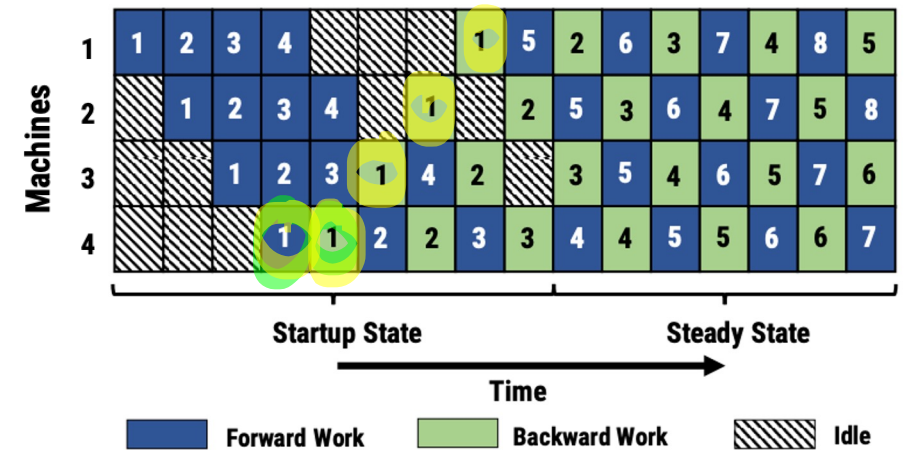
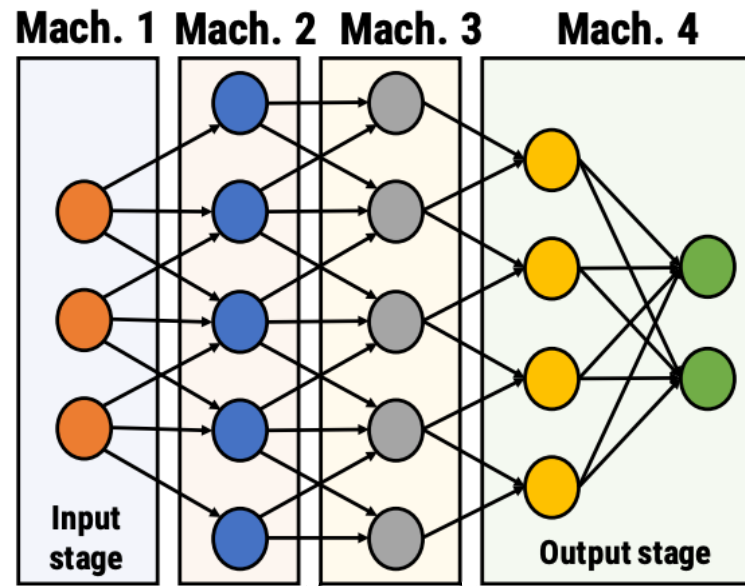
- The previous examples of tensor parallelism also implicitly leverage *pipeline parallelism*, where different layers or modules are put on different GPUs
- Issue: the computation across layers is inherently **sequential**, not parallel!



- Idea: work on multiple *microbatches* concurrently!

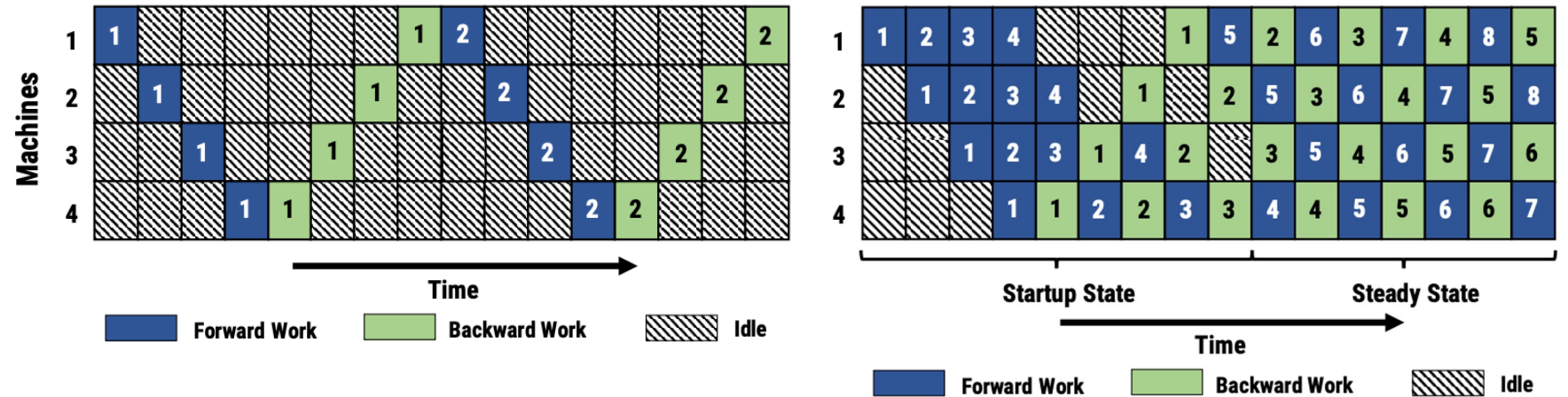
# Pipeline Parallelism

- The previous examples of tensor parallelism also implicitly leverage *pipeline parallelism*, where different layers or modules are put on different GPUs
- Issue: the computation across layers is inherently **sequential**, not parallel!



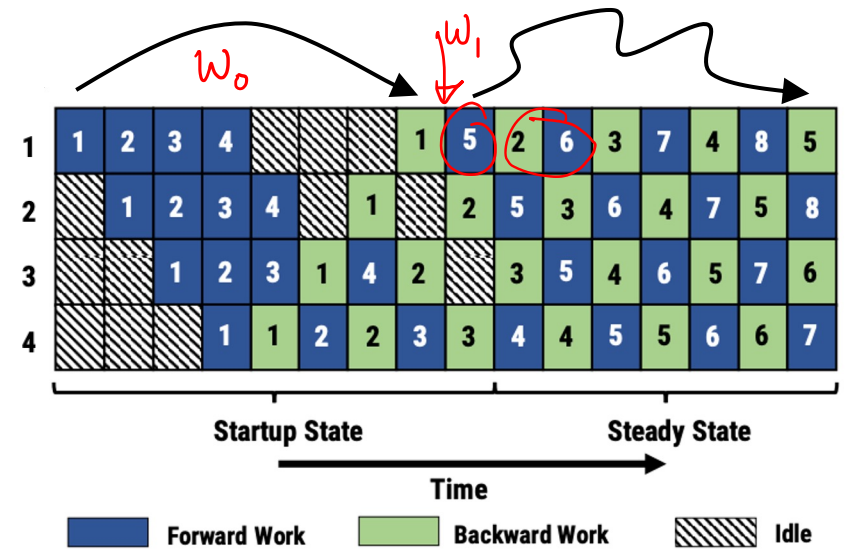
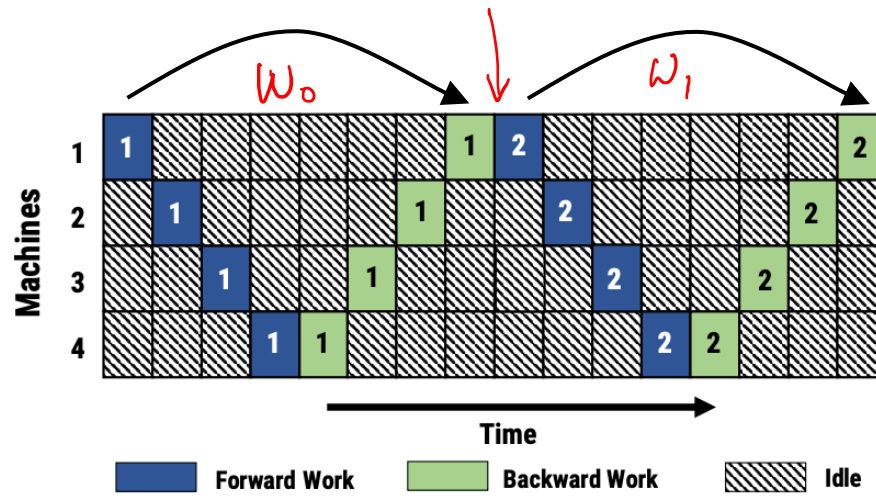
- Idea: work on multiple *microbatches* concurrently!

# Pipeline Parallelism



- The schedule above-right is the one-forward-one-backward (1F1B) mechanism: in the steady state, each GPU alternates between one forward pass and one backward pass

# Pipeline Parallelism



- Issue: if weights are updated in every backward pass, the weights used to compute the forward pass for a microbatch can be different from the weights used to compute the backward pass
  - The divergence is worse for earlier GPUs in the pipeline
  - Can lead to poor model convergence/optimization
- Solution: weight stashing – after every forward pass, store the weights and reload them for the corresponding backward pass

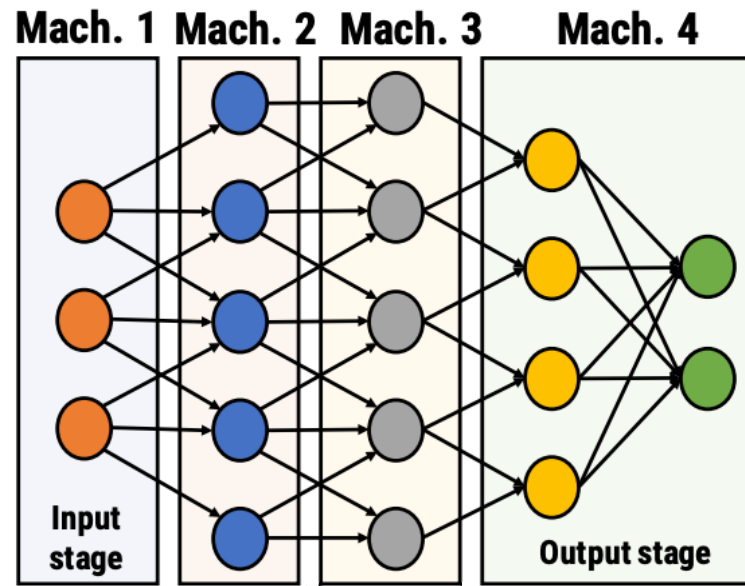
# Pipeline Parallelism



- Indicates a pipeline flush between minibatches; the sync is required to compute the parameters for the next minibatch's forward pass
- The 1F1B is still highly efficient even when forward and backward passes take different amounts of compute (as is typically the case)

# Pipeline Parallelism

- Another interesting question that arises with pipeline parallelism: how should we partition the layers across GPUs?
- Solution: profile the code and apply dynamic programming!





# Pipeline Parallelism

- Another interesting question that arises with pipeline parallelism: how should we partition the layers across GPUs?
- Solution: profile the code and apply dynamic programming!

Let  $A(j, m)$  denote the time taken by the slowest stage in the optimal pipeline between layers 1 and  $j$  using  $m$  machines. The goal of our algorithm is to find  $A(N, M)$ , and the corresponding partitioning. Let  $T(i \rightarrow j, m)$  denote the time taken by a single stage spanning layers  $i$  through  $j$ , replicated over  $m$  machines.

$$T(i \rightarrow j, m) = \frac{1}{m} \max \left( \sum_{l=i}^j T_l, \sum_{l=i}^j W_l^m \right)$$

where the left term inside the max is the total computation time for all the layers in the stage, and the right term is the total **communication time** for all the layers in the stage.

The optimal pipeline consisting of layers from 1 through  $j$  using  $m$  machines could either be a single stage replicated  $m$  times, or be composed of multiple stages.

**Case 1:** The optimal pipeline contains only one stage, replicated  $m$  times. In this case,

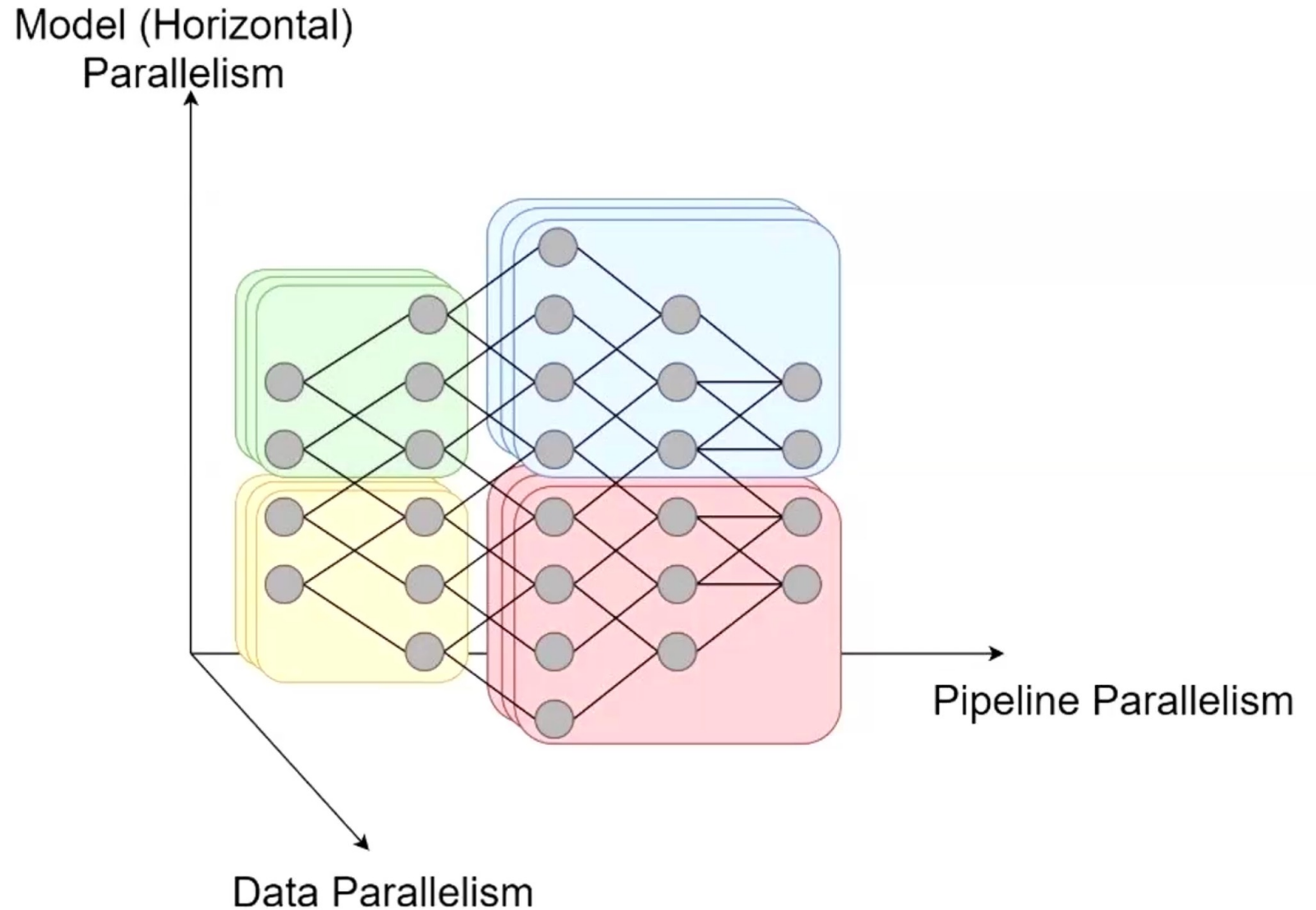
$$A(j, m) = T(1 \rightarrow j, m)$$

**Case 2:** The optimal pipeline contains more than one stage. In this case, it can be broken into an optimal sub-pipeline consisting of layers from 1 through  $i$  with  $m - m'$  machines followed by a single stage with layers  $i + 1$  through  $j$  replicated over  $m'$  machines. Then, using the optimal sub-problem property, we have

$$A(j, m) = \min_{1 \leq i < j} \min_{1 \leq m' < m} \max \begin{cases} A(i, m - m') \\ 2 \cdot C_i \\ T(i + 1 \rightarrow j, m') \end{cases}$$

The story so far:

Ultimately, data parallelism is still king and tensor / pipeline parallelism are used to support pushing more data through the forward and backward passes







# Optimizer Parallelism

# Optimizer Parallelism

- If we only need to compute/store the parameters and gradients, then data + model + pipeline parallelism (typically) suffices
  - E.g., for mini-batch SGD, the update only requires the gradients and the current weights:

$$W^{(t+1)} \leftarrow W^{(t)} - \gamma \nabla \ell^{(B)}(W^{(t)})$$

- However, many advanced optimization algorithms require storing additional intermediate or *state variables* to perform the parameter update

# Adam (Adaptive Moment Estimation)

- High-level intuition: Adam combines SGD with momentum (memory of previous gradient steps) and RMSProp (scaled step sizes based on previous gradients)

$$W^{(t+1)} \leftarrow W^{(t)} - \frac{\gamma}{\sqrt{S_t / (1 - \beta_2^t)}} \odot \left( \frac{M_t}{1 - \beta_1^t} \right)$$

where

- $M_t = \beta_1 M_{t-1} + (1 - \beta_1) \nabla \ell^{(B)}(W^{(t)})$
- $S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left( \nabla \ell^{(B)}(W^{(t)}) \odot \nabla \ell^{(B)}(W^{(t)}) \right)$
- $\beta_1$  and  $\beta_2$  are *decay* parameters that dictate how much  $M_t$  and  $S_t$  are defined by previous time steps
- $M_{-1}$  and  $S_{-1}$  are initialized to matrices of all zeros

# Adam (Adaptive Moment Estimation)

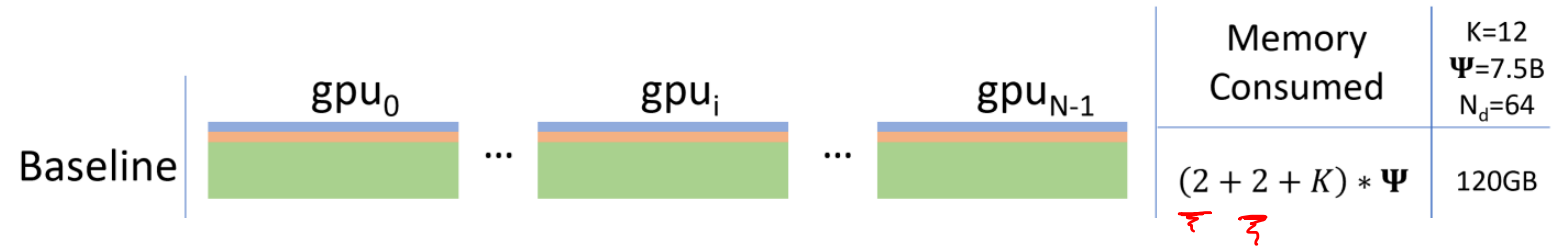
- High-level intuition: Adam combines SGD with momentum (memory of previous gradient steps) and RMSProp (scaled step sizes based on previous gradients)

$$W^{(t+1)} \leftarrow W^{(t)} - \frac{\gamma}{\sqrt{S_t / (1 - \beta_2^t)}} \odot \left( \frac{M_t}{1 - \beta_1^t} \right)$$

where

- $M_t = \beta_1 M_{t-1} + (1 - \beta_1) \nabla \ell^{(B)}(W^{(t)})$
- $S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left( \nabla \ell^{(B)}(W^{(t)}) \odot \nabla \ell^{(B)}(W^{(t)}) \right)$
- Mixed-precision training:  $M_t$  and  $S_t$  are the same dimensionality as  $W^{(t)}$  and are typically stored in FP32 (instead of FP16) because of the squared term in  $S_t$

# Optimizer Parallelism



- For Adam,  $K = 12 = 3 * 4$  because each parameter in  $\nabla l(W^{(t)})$ ,  $M_t$ , and  $S_t$  is stored in FP32 = 4 bytes per parameter

■ Parameters   ■ Gradients   ■ Optimizer States

Figure 1: Comparing the per-device memory consumption of model states, with three stages of ZeRO-DP optimizations.  $\Psi$  denotes model size (number of parameters),  $K$  denotes the memory multiplier of optimizer states, and  $N_d$  denotes DP degree. In the example, we assume a model size of  $\Psi = 7.5B$  and DP of  $N_d = 64$  with  $K = 12$  based on mixed-precision training with Adam optimizer.

# Optimizer Parallelism

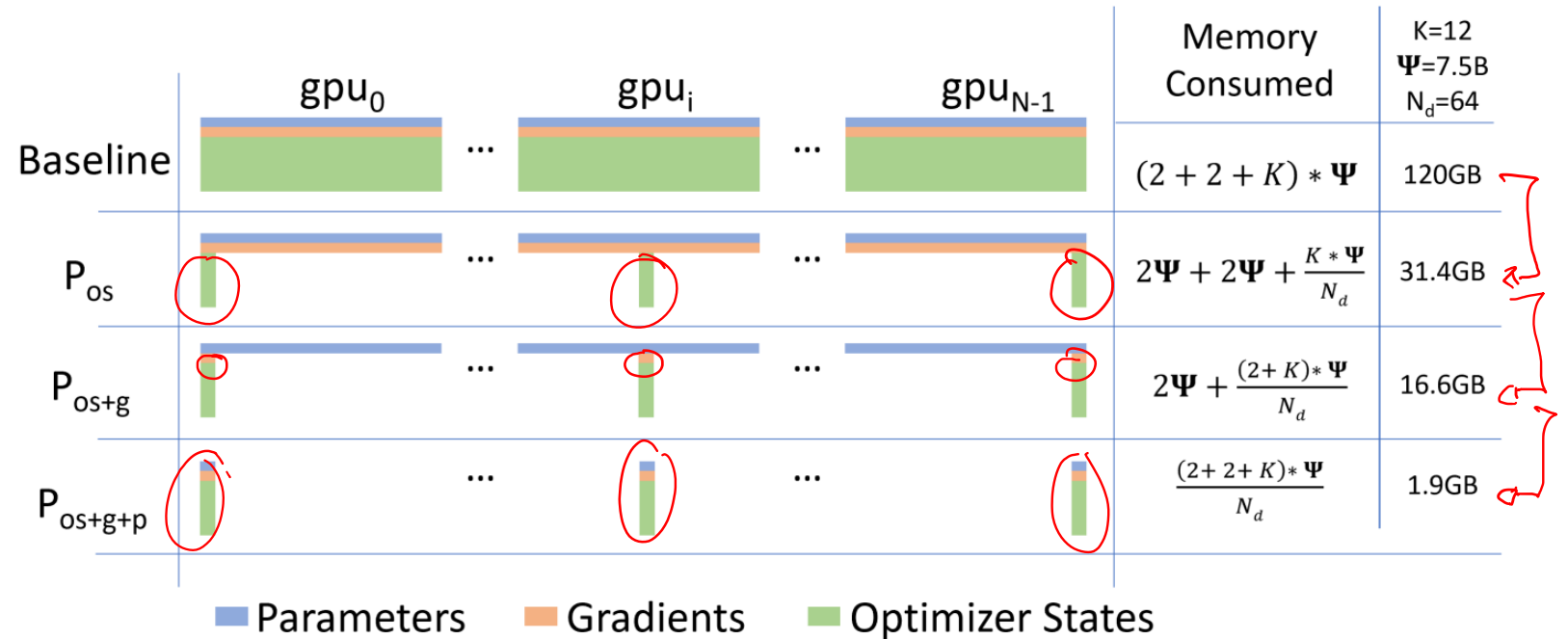


Figure 1: Comparing the per-device memory consumption of model states, with three stages of ZeRO-DP optimizations.  $\Psi$  denotes model size (number of parameters),  $K$  denotes the memory multiplier of optimizer states, and  $N_d$  denotes DP degree. In the example, we assume a model size of  $\Psi = 7.5B$  and DP of  $N_d = 64$  with  $K = 12$  based on mixed-precision training with Adam optimizer.

(ZeRO-DP = Zero redundancy optimizer powered data parallelism)

# Optimizer Parallelism

| DP          | 7.5B Model (GB) |             |              | 128B Model (GB) |            |              | 1T Model (GB) |            |              |
|-------------|-----------------|-------------|--------------|-----------------|------------|--------------|---------------|------------|--------------|
|             | $P_{os}$        | $P_{os+g}$  | $P_{os+g+p}$ | $P_{os}$        | $P_{os+g}$ | $P_{os+g+p}$ | $P_{os}$      | $P_{os+g}$ | $P_{os+g+p}$ |
| 1           | 120             | 120         | 120          | 2048            | 2048       | 2048         | 16000         | 16000      | 16000        |
| 4           | 52.5            | 41.3        | 30           | 896             | 704        | 512          | 7000          | 5500       | 4000         |
| 16          | 35.6            | <b>21.6</b> | 7.5          | 608             | 368        | 128          | 4750          | 2875       | 1000         |
| 64          | <b>31.4</b>     | 16.6        | 1.88         | 536             | 284        | <b>32</b>    | 4187          | 2218       | 250          |
| 256         | 30.4            | 15.4        | 0.47         | 518             | 263        | 8            | 4046          | 2054       | 62.5         |
| <u>1024</u> | 30.1            | 15.1        | 0.12         | 513             | 257        | 2            | 4011          | 2013       | <b>15.6</b>  |

Table 1: Per-device memory consumption of different optimizations in *ZeRO*-DP as a function of DP degree . Bold-faced text are the combinations for which the model can fit into a cluster of 32GB V100 GPUs.

## 5.4 Implication on Model Size

The three phases of partitioning  $P_{os}$ ,  $P_{os+g}$ , and  $P_{os+g+p}$  reduces the memory consumption of each data parallel process on model states by up to 4x, 8x, and  $N_d$  respectively. Table 1 analyzes model-state memory consumption of a few example models under the 3 stages of *ZeRO*-DP optimizations for varying DP degree. Without *ZeRO*, the memory consumption is equal to the first row in the table, regardless of the DP degree. Note that, with  $N_d = 64$ , *ZeRO* can train models with up to 7.5B, 14B, and 128B parameters using  $P_{os}$ ,  $P_{os+g}$ , and  $P_{os+g+p}$ , respectively. When  $N_d = 1024$ , *ZeRO* with all of its optimizations enabled ( $P_{os+g+p}$ ) could train models with **1 TRILLION parameters!** Or potentially, models with ARBITRARY size! Without *ZeRO*, the largest model DP alone can run has less than 1.5 Billion parameters.

# Optimizer Parallelism



The three phases of partitioning  $P_{os}$ ,  $P_{os+g}$ , and  $P_{os+g+p}$  reduces the memory consumption of each data parallel process on model states by up to 4x, 8x, and  $N_d$  respectively. Table 1 analyzes model-state memory consumption of a few example models under the 3 stages of *ZeRO*-DP optimizations for varying DP degree. Without *ZeRO*, the memory consumption is equal to the first row in the table, regardless of the DP degree. Note that, with  $N_d = 64$ , *ZeRO* can train models with up to 7.5B, 14B, and 128B parameters using  $P_{os}$ ,  $P_{os+g}$ , and  $P_{os+g+p}$ , respectively. When  $N_d = 1024$ , *ZeRO* with all of its optimizations enabled ( $P_{os+g+p}$ ) could train models with **1 TRILLION parameters!** Or potentially, models with ARBITRARY size! Without *ZeRO*, the largest model DP alone can run has less than 1.5 Billion parameters.



# Recall: How large are LLMs?

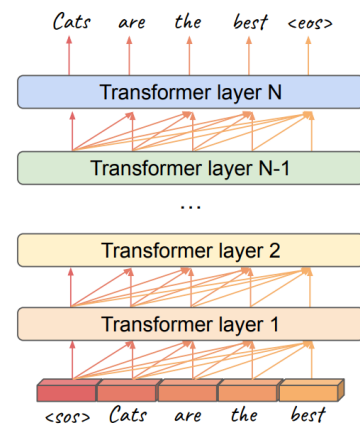
| Model                  | Creators | Year of release | Training Data (# tokens) | Model Size (# parameters)          |
|------------------------|----------|-----------------|--------------------------|------------------------------------|
| GPT-2                  | OpenAI   | 2019            | ~10 billion (40Gb)       | 1.5 billion                        |
| GPT-3<br>(cf. ChatGPT) | OpenAI   | 2020            | 300 billion              | 175 billion                        |
| PaLM                   | Google   | 2022            | 780 billion              | 540 billion                        |
| Chinchilla             | DeepMind | 2022            | 1.4 trillion             | 70 billion                         |
| LaMDA<br>(cf. Bard)    | Google   | 2022            | 1.56 trillion            | 137 billion                        |
| LLaMA                  | Meta     | 2023            | 1.4 trillion             | 65 billion                         |
| LLaMA-2                | Meta     | 2023            | 2 trillion               | 70 billion                         |
| GPT-4                  | OpenAI   | 2023            | ?                        | ? (1.76 trillion)                  |
| Gemini (Ultra)         | Google   | 2023            | ?                        | ? (1.5 trillion)                   |
| LLaMA-3                | Meta     | 2024            | 15 trillion              | 405 billion                        |
| LLaMA-4                | Meta     | 2025            | 30 trillion              | 2 trillion<br>(288 billion active) |

# Parallelism in LLM Training

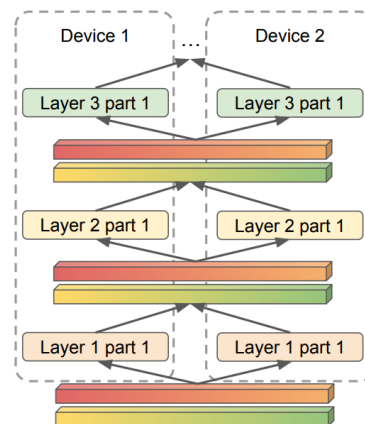
- Goal: divide the work of training an LLM across multiple GPUs such that inter-GPU communication is minimized
- Good news: Transformer-based LLM architectures are highly parallelizable!
- Easily exploitable parallelism in LLM training includes:
  - Data parallelism
  - Model or tensor parallelism
  - Pipeline parallelism
  - Optimizer-based parallelism
  - **Token parallelism**
  - **Expert parallelism**

# Token Parallelism

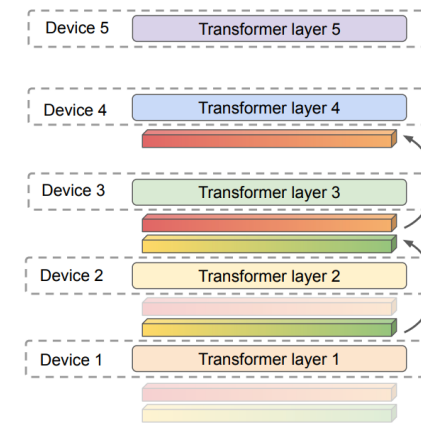
- Insight: in a transformer LM with causal attention, the computation for each *token* in some transformer block only depends on the previous tokens at that layer
- Idea: instead of waiting for the entire previous layer to finish, start working on token  $t$  in layer  $l$  as soon as tokens 1 through  $t - 1$  in layer  $l - 1$  are done



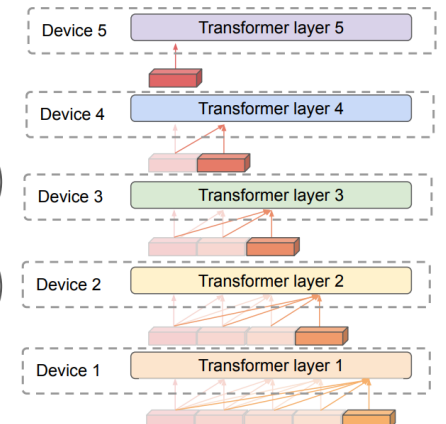
(a) Transformer-based LM



(b) Operation partitioning (Megatron-LM)



(c) Microbatch-based pipeline parallelism (GPipe)

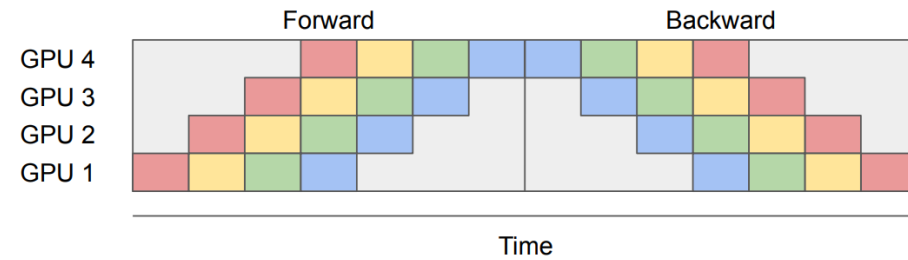


(d) Token-based pipeline parallelism (TeraPipe)

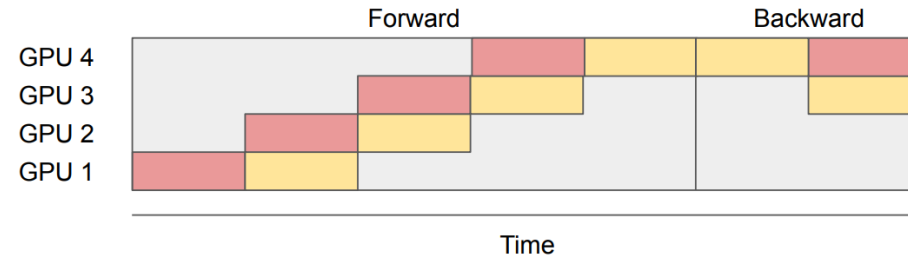
# Token Parallelism

- Insight: in a transformer LM, the computation for each *token* in some transformer block only depends on the previous tokens at that layer
- Idea: instead of waiting for the entire previous layer to finish, start working on token  $t$  in layer  $l$  as soon as tokens 1 through  $t - 1$  in layer  $l - 1$  are done
- Intuition: increasing pipeline granularity reduces idle time in the pipeline!
  - Efficiency gains scale with sequence lengths and models have been moving towards longer contexts

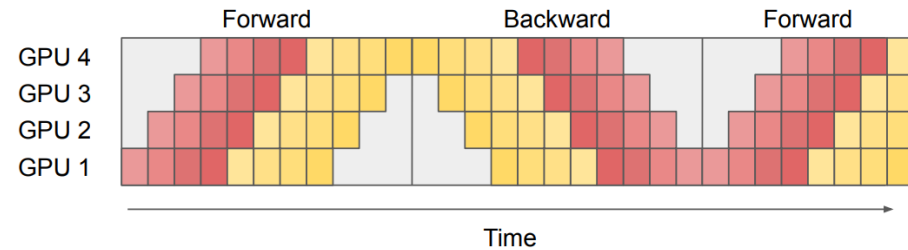
# Token Parallelism



(a) Microbatch-based pipeline parallelism



(b) Microbatch-based pipeline parallelism with small batch size



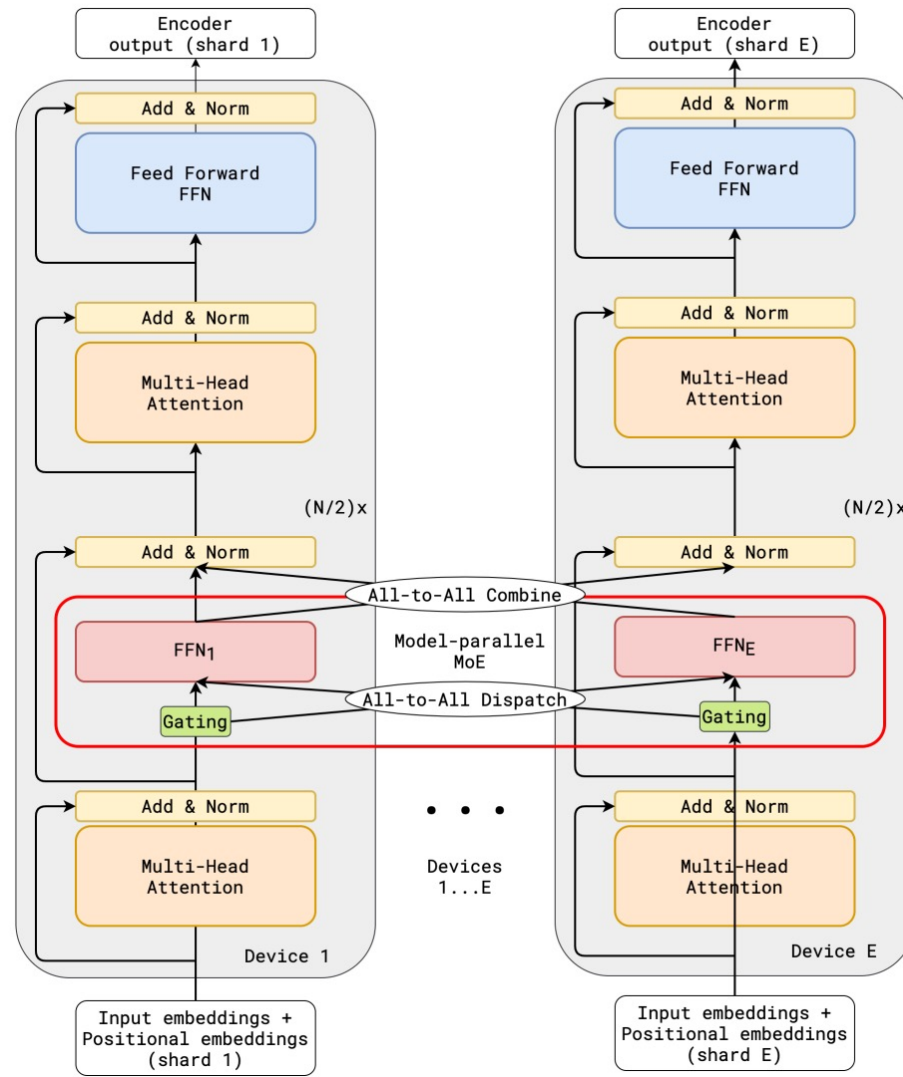
(c) TeraPipe

- Note: this figure shows an all-forward-all-backward mechanism, which eliminates the need for weight stashing but introduces more idle time

- Intuition: increasing pipeline granularity reduces idle time in the pipeline!
- Efficiency gains scale with sequence lengths and models have been moving towards longer contexts

# Expert Parallelism

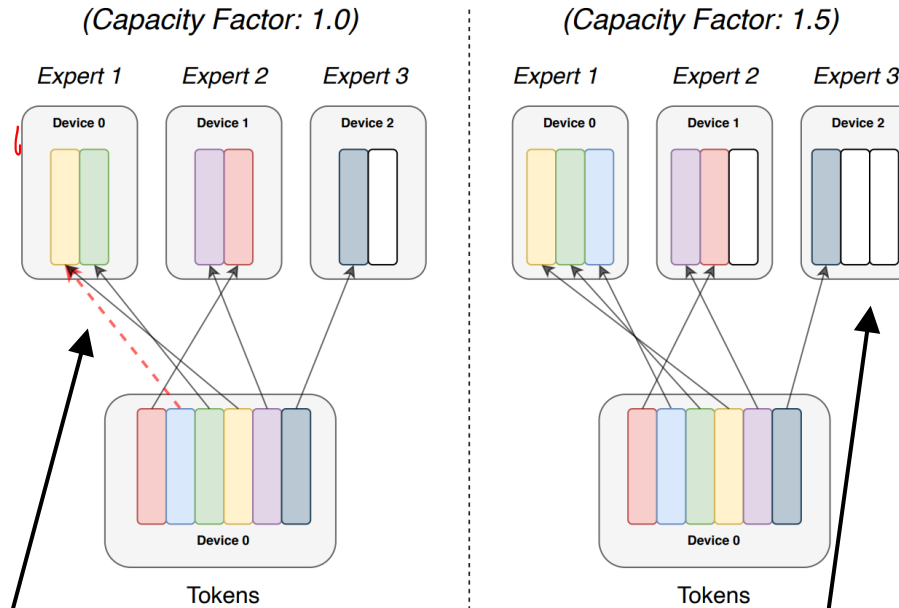
- Approach: just put each expert on a different GPU!



- Key consideration: to enforce a balanced load across GPUs, each expert is assigned a *capacity*
  - If an expert is assigned more tokens than its capacity, those tokens are just passed directly to the next layer via residual connections

# Expert Parallelism

- Approach: just put each expert on a different GPU!



- If the capacity factor is too small, overallocation is likely
- If the capacity factor is too large, some GPUs will likely be underutilized

- Key consideration: to enforce a balanced load across GPUs, each expert is assigned a *capacity*

- If an expert is assigned more tokens than its capacity, those tokens are just passed directly to the next layer via residual connections